Highlights

A Dual-System Enhanced Large Language Model Architecture for Real-Time Joint-Operation Mission Planning in Command and Control Systems

Changrui Zhang, Chengwei Yang

- A dual-system LLM architecture for real-time joint-operation mission planning in command-and-control systems
- Evidence-grounded loop coupling case memory, adaptive capability control, and reflection repair
- Full loop improves mission success over pure-LLM baseline (paired $t(4) = 3.75$, $p = 0.02$)
- Three-tier response: 5.6 ms emergency plan, 22.8 s delta refinement to 96.4% quality, minutes-level full optimization
- Native DoDAF OV-5b/OV-6c and SISO C2SIM exports as a standard-compliant interoperability hub

A Dual-System Enhanced Large Language Model Architecture for Real-Time Joint-Operation Mission Planning in Command and Control Systems

Changrui Zhang[a], Chengwei Yang[a,*]

[a]*Beijing Institute of Technology, Beijing, China*

Abstract

Modern multi-domain joint operations demand planning under partial observability, adversarial conditions, tight time windows, and coupled multi-domain competing objectives, where decision cycles are measured in seconds. Prompt-only large language model (LLM) assistants lag on three fronts: aligning generated courses of action with tactical constraints, resisting the variance drift of autoregressive full regeneration, and producing a plan early enough to inform a command-and-control (C2) decision. This paper presents a dual-system enhanced LLM architecture that addresses these constraints as an integrated planning loop.

The pipeline couples an episodic case-memory module, a stochastic-differential-equation (SDE)-driven adaptive capability controller, and a reflection loop into one traceable system. The memory module reuses historical operation experience as positive/negative priors; the SDE controller projects scenario attributes (terrain, weather, electronic-warfare threat, time pressure) onto a bounded capability-weight state through a bottleneck-aware stochastic update; and reflection converts simulator feedback into local prompt repairs. Plans are generated, evaluated, and exported into Department of Defense Architecture Framework (DoDAF) OV-5b/OV-6c and C2SIM artifacts. On a reference scenario with 10 iterations and 50 Monte Carlo runs, the full loop raises mission success from 62.75 to 65.76 and overall effectiveness from 73.84 to 78.75 over the pure-LLM baseline; a five-seed replication confirms the gain is statistically significant ($t(4) = 3.75$,

*Corresponding author.

Email address: yangchengwei@bit.edu.cn (Chengwei Yang)

$p = 0.02$, $d = 1.68$). Across five scene-out scenarios the full pipeline beats the pure baseline in four of five.

For real-time use, a decoupled dual-system path is introduced. A rule-based emergency tier emits a complete plan and its DoDAF/C2SIM export in milliseconds (5.6 ms, 1.4 ms warm) with no LLM call; a bottleneck-driven delta-patching tier refines that anchor to 96.4% of the full-loop mission-success quality within 22.8 s, against 365 s for the complete loop. This yields a three-tier latency profile—millisecond emergency, tens-of-seconds tactical refinement, minutes-level full optimization. The delta-patching mechanism preserves the high-quality anchor that full regeneration destroys, resolving the variance regression inherent in autoregressive full-plan rewriting. The pipeline does not replace physics-based simulators (e.g., OneSAF, FLAMES); it emits IEEE 1516/SISO C2SIM-compatible artifacts and acts as a standard-compliant planning hub linking LLM courses of action into existing C2 systems and distributed simulation federations.

Keywords: military command and control, large language models, mission planning, case-based reasoning, adaptive weighting, kill-chain optimization, C2SIM interoperability, real-time replanning

---

1. Introduction

Recent transformer and foundation-model advances have made large language models (LLMs) increasingly capable of flexible multi-step planning and decision support, which makes them attractive for simulation-centric scenario exploration workflows [1–8]. In the military domain this trend is particularly consequential. Recent work has introduced LLM-based command agents that reconstruct warfare simulation and command decision-making, treating the LLM as a commander that interprets a scenario, issues orders, and reacts to the simulated battle state [9]; complementary efforts have deployed LLM agents in tactical wargaming and as reasoning front-ends for battle-field decision support [10, 11], and new benchmarks have been proposed to assess LLM readiness for military decision-making under mission constraints [12]. Collectively, this line of work establishes that LLMs can serve as credible planners and decision-support engines inside operational simulation. Reasoning-time performance can be further shaped by prompt-side strategies such as chain-of-thought, self-consistency, ReAct-style tool interaction, deliberate search over thought branches, and explicit tool use

[13–17]. At the same time, planning quality remains sensitive to context selection, prior experience, and iterative self-correction. Evidence from the current evaluation suggests that three design directions are central to its implementation: retrieval-style memory augmentation, adaptive capability weighting, and reflection-driven prompt adjustment.

The memory component is closer to case-based reasoning (CBR) and retrieval-augmented generation than to standalone prompting. The implemented system stores structured cases, retrieves them through a FAISS-backed semantic index when available, and falls back to keyword retrieval otherwise [18–21]. In broader architectural terms, that places it closer to external-memory and memory-network families [22–24] and to classical CBR formulations [25, 26]. The reflection side is closer to iterative self-improvement and agent-memory schemes, including Reflexion, Self-Refine, generative-agent memory, prompt optimization, and the recent Memento line, although the current implementation should still be interpreted strictly as a system-specific planning loop rather than as a reimplementation of any single external framework [27–31].

The proposed pipeline also differs from several prominent LLM-based planning paradigms in important ways. Unlike RAP-style reasoning-via-planning methods that use an LLM as a world model to simulate state transitions [32], the present system relies on an external, domain-structured simulator. Unlike LLM-Planner and SayCan, which ground language-model outputs in executable robot actions through affordance functions [33, 34], this pipeline grounds plans in a typed, capability-dimension-rich military simulation model that evaluates plans holistically. Unlike LLM+P and PlanBench, which focus on classical planning formalisms (PDDL, Blocksworld) and evaluate LLMs as standalone planners [35, 36], the present work treats the LLM as one component in a broader feedback loop that includes external memory, adaptive weighting, and simulation-driven critique. Unlike Tree-of-Thoughts and Graph-of-Thoughts approaches that expand the LLM's internal search budget [16, 37], this pipeline offloads search to the simulator and uses reflection as a repair signal rather than a branching mechanism. These architectural differences reflect a deliberate design choice to keep the LLM's role bounded and to concentrate domain knowledge in the memory, controller, and simulator components rather than in the LLM's parametric knowledge.

Within the proposed framework, the target problem is not free-form text generation. Instead, the system aims to produce structured multi-domain

operation plans that can be simulated, compared under ablation settings, and exported to architecture and interoperability artifacts. Throughout this manuscript, the term "joint-operation" is used in a domain-level sense to refer to coordinated actions across land, maritime, air, and cyber/electronic-warfare domains, rather than in the narrower institutional sense of multi-service joint command (e.g., Joint Chiefs of Staff). This usage reflects the seven-capability-dimension representation (fires, mobility, protection, awareness, EW, sustainment, C2) that spans multiple operational domains within the proposed simulation framework. The five generalization scenarios (coastal assault, urban defense, mountain reconnaissance, river crossing, and maritime sustainment) each engage a different subset of these domains and capability dimensions, and the pipeline is designed to handle this diversity through domain-agnostic structured representations rather than service-specific doctrines. The simulation layer scores plans through a five-stage kill-chain abstraction, computes mission-level metrics such as mission success and overall effectiveness, and aggregates repeated stochastic runs into Monte Carlo summaries. At an operational-reading level, the five stages loosely map to target discovery, disruption and suppression, breach of defended nodes, control of key objectives, and sustainment of tempo and support. This mapping is intended only as a theory-alignment aid: the implemented simulator is still an internal simulation abstraction inspired by joint-targeting, F2T2EA, and network-centric command thinking rather than a doctrinal one-to-one reproduction [38–41]. The export layer produces DoDAF and C2SIM outputs aligned with the system's structured plan objects and broader simulation-interoperability practice [42–45].

These capabilities are motivated by three concrete pain points that recur in military command-and-control (C2) practice. First, *tactical-experience reuse*: a commander's accumulated operational experience is fragmented across the force, and without an efficient mechanism it cannot be distilled into reusable positive/negative priors for a new mission; the Memento case bank addresses precisely this reuse problem. Second, *dynamic capability balancing under threat*: in a rapidly shifting electromagnetic and time-constrained environment, a fixed doctrinal weighting of fire, mobility, and communication capabilities is too rigid, so the HOPE controller adaptively re-weights capabilities in response to scenario attributes and simulated feedback. Third, *response latency*: full-regeneration planning is too slow and, worse, is variance-bound—regenerating an entire plan from scratch destroys the high-quality partial solution already found, so a local, diagnosis-driven patch

that preserves the anchor is the only mechanism that can both converge monotonically and meet a real-time decision budget. The dual-system architecture below is designed around these three constraints.

From a research-positioning perspective, the contributions are not the introduction of a single new component but the identification and resolution of two phenomenon-level problems that arise specifically when an LLM is used as a military C2 planner. First, we show empirically that *full autoregressive regeneration is variance-regressive*: when a high-quality plan already exists, rewriting the entire plan from scratch destroys that anchor and the trajectory collapses toward the sampling mean. Second, we show that *local, diagnosis-driven patching is monotone*: a delta-patching loop that rewrites only the bottleneck phase, gated by simulation feedback, converges to the best-so-far anchor without regression. These two observations are non-trivial in the C2 setting because the kill chain imposes a strict temporal/causal dependency order and because enabling resources (protection, C2, sustainment) are zero-sum across phases, so a planner must hold a good partial solution while improving a single weak link. The practical consequence is a decoupled dual-system architecture delivering the three-tier latency profile that a live C2 decision cycle requires. The contribution is therefore this phenomenon-driven design and its measured behaviour, situated within a standard-compliant planning hub that connects LLM-generated courses of action to DoDAF/C2SIM downstream environments.

This work intentionally stays close to what can be verified from files in the current implementation. It does not claim formal guarantees, external benchmark coverage, or a complete literature survey. Instead, the paper makes five application-oriented contributions grounded in code and experiment outputs:

1. It documents the proposed planning pipeline as an integrated workflow that combines case retrieval, adaptive weighting, bottleneck detection, reflection, a decoupled dual-system fast/slow response hierarchy for real-time mission planning, structured simulation evaluation, and standards-oriented exporters.
2. It reports a single-scenario five-way ablation (Pure LLM, +Memento, +Hope, +Reflection, Full) under default HOPE parameters, plus a multi-seed replication across five seeds confirming statistical significance for the default Full setting ($t(4) = 3.75$, $p = 0.02$, $d = 1.68$).
3. It reports a five-scenario cross-scenario transfer study characterizing where gains are stable, where they remain scenario-dependent, and

where the pipeline shows residual limitations.

4. It provides a Reflection-only single-scenario ablation (and a three-seed Reflection-only replication on the earlier hardware suite), revealing that standalone reflection degrades or does not improve plan quality and should be deployed only within the coupled architecture.
5. It validates the pipeline across three LLM backends (DeepSeek-R1, Qwen3.5, Gemma-4), supplemented by external baselines (few-shot CoT, random search, single-pass Memento), a face validity sensitivity analysis, and hyperparameter and case-bank-size ablations.

The rest of the paper describes the implemented pipeline, the selected evidence scope, the main quantitative findings, and the current threats to validity that still limit stronger claims.

2. Method

The proposed framework is organized around structured scenario, plan, simulation, and export objects rather than free-form prompting alone. This section deliberately limits itself to mechanisms that are directly supported by the current code and stored outputs.

2.1. Pipeline overview

Figure 1 presents the overall framework of the implemented planning pipeline. The framework starts from a structured scenario representation and couples two auxiliary mechanisms before each generation step: a Memento memory module that retrieves and filters prior cases, and a Hope adaptive controller that transforms scenario attributes into a bounded capability state. These two information sources, together with optional reflection patches from the previous iteration, are fused during prompt construction for LLM-based structured plan generation. The resulting candidate plan is then evaluated by the simulation layer through the system's five-stage kill-chain abstraction, which yields mission-level metrics and stage-level deficits. These evaluation signals are used in two feedback paths: one path updates the adaptive capability state, and the other path produces reflection guidance for the next generation round. After the iterative loop terminates, the highest-scoring plan is selected and exported into the downstream structured artifacts used by the system.

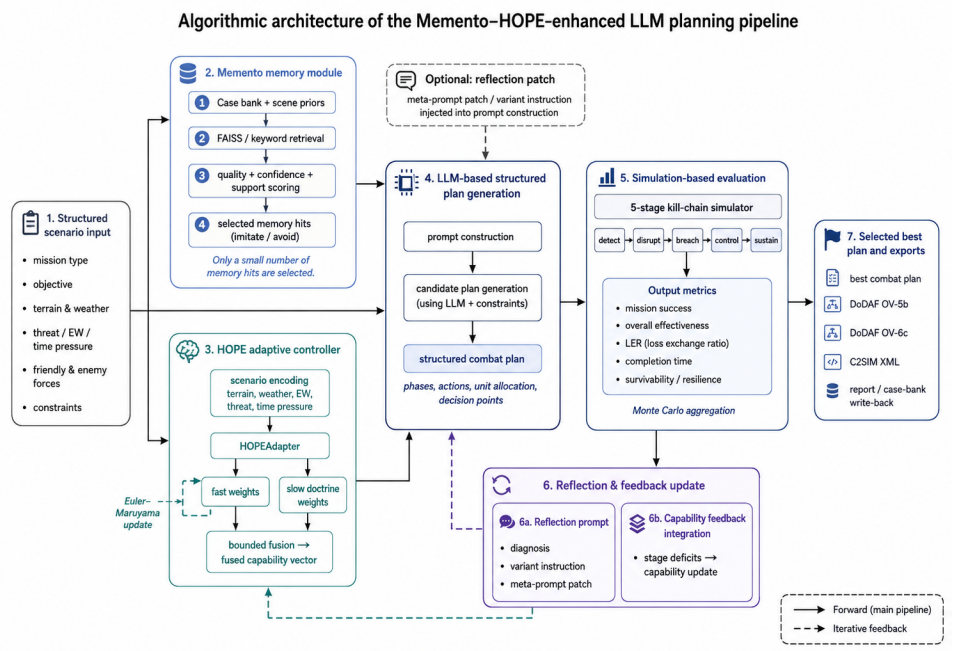

Figure 1: Overall framework of the Memento–HOPE-enhanced LLM planning pipeline. The framework maps structured scenario input to four tightly connected stages: memory retrieval and selection, adaptive capability modulation, LLM-based structured plan generation, and simulation-based evaluation. The solid arrows denote the main forward computation from scenario encoding to best-plan selection and export, whereas the dashed arrows denote the iterative feedback paths that inject reflection guidance and capability updates into subsequent planning rounds.

2.2. Structured scenario input and plan representation

The domain layer defines normalized capability dimensions (`fires`, `mobility`, `protection`, `awareness`, `ew`, `sustainment`, and `c2`), typed actions, force units, scenarios, plan phases, and complete combat plans. These objects enforce non-empty text fields, bounded numeric ratios, and structured phase/action serialization. The downstream simulator and exporters therefore operate on typed fields rather than on unconstrained narrative text.

2.3. Memory module: case retrieval, selection, and memory-conditioned weight initialization

The case-memory module stores records with mission type, terrain, objective, roles, key actions, lessons, outcomes, and metric summaries. When

optional embedding dependencies are available, the module uses a Sentence-Transformer encoder with a FAISS index; otherwise it falls back to keyword retrieval [18, 20, 21]. At the design level, that retrieval path remains closer to case-based reasoning than to standalone prompting [25, 26]. Retrieved items are scored by confidence, blended with a case-quality heuristic, and labeled as either *imitate* or *avoid* according to the source outcome.

The pipeline does not pass all retrieved cases directly to the generator. Instead, it computes a scenario-dependent support score, sorts candidates by support, quality, and confidence, and keeps at most two selected memory hits in strict mode. In the current implementation, this two-hit cap is an engineering prompt-budget heuristic rather than a theorem-backed optimum: it is meant to reduce prompt crowding and limit negative transfer from weakly matched cases while still preserving at least one positive and one cautionary cue when both are available. For sparse cross-scenario transfer settings, the pipeline can also inject hand-authored scene priors for coastal assault, mountain reconnaissance, maritime sustainment, and urban defense scenes. These priors are represented as standard case records and enter the same scoring and selection path.

From a data-lifecycle perspective, the implemented memory path is a four-step loop: offline case curation, line-delimited JSONL persistence, online retrieval and reranking, and optional write-back behind a quality gate. Each stored `CaseRecord` keeps a compact but typed field set. The mission-context fields are `case_id`, `mission_type`, `terrain`, `objective`, `friendly_roles`, and `enemy_roles`; the action-and-outcome fields are `key_actions`, `lessons`, `outcome`, `tags`, and `metrics`; and the maintenance fields are `update_count`, `last_updated`, and `source_scenarios`. The system's data-loading routine reads the JSONL bank one line at a time, `retrieve()` chooses the FAISS path when semantic backends are available and otherwise falls back to keyword similarity, and the pipeline-level `_select_memory_hits()` stage reorders candidates by support score, quality, and confidence before truncation. If write-back is enabled, `_write_back_case()` creates a candidate record only after the simulated plan clears mission-success, effectiveness, and minimum-stage thresholds, and `upsert_record()` then merges or appends the result inside the same JSONL store. For the canonical ablation and generalization evidence used in this paper, that final write-back step is disabled in the recorded configurations, so the present manuscript should be read as documenting the memory read-and-use path rather than an online continual-learning experiment.

Memory-conditioned weight initialization.. In the original design the Hope fast-weight target $\hat{\mathbf{W}}_{\text{fast}}$ depends solely on scenario attributes (terrain, weather, threat coefficients) through the `HOPEAdapter` network. The retrieved case memory only influences the LLM prompt, not the capability-weighting path. A lightweight coupling mechanism is introduced to integrate retrieved case metrics into the fast-weight initialization path: when the Memento module retrieves cases whose stored stage-level metrics indicate a specific weakness (e.g., the most similar historical case failed badly at `breach`), a memory correction vector nudges the fast-weight target toward the capabilities that feed that weak stage.

Formally, let $\mathcal{M} = \{\mathbf{m}_1, \ldots, \mathbf{m}_k\}$ be the stage-score vectors of the k retrieved cases. For each case i and each stage j, if $m_{i,j} < 0.55$, we accumulate a correction proportional to the deficit and the stage-to-capability mapping:

$$\mathbf{r}_{\text{mem}} = \sum_{i=1}^{k} \sum_{j \in \mathcal{S}} \mathbb{1}[m_{i,j} < 0.55] \cdot (1 - m_{i,j}) \cdot 0.18 \cdot \mathbf{c}_j,$$

where $\mathcal{S}$ is the set of five kill-chain stages and $\mathbf{c}_j$ is the stage-to-capability mapping for stage j. The correction is clamped to $[-0.18, 0.28]$ and blended with the adapter output: $\hat{\mathbf{W}}_{\text{fast}} \leftarrow \hat{\mathbf{W}}_{\text{fast}} + 0.30\,\mathbf{r}_{\text{mem}}$. This mechanism is intentionally lightweight—it does not require retraining the adapter or modifying the case bank schema—and it means that the pipeline's two auxiliary modules (memory and adaptive weighting) are no longer purely serial but share information through a common capability representation.

2.4. SDE-driven adaptive capability controller: adaptive weighting, bottleneck detection, and capability feedback

The adaptive controller (referred to here as the *adaptive capability controller*) maintains a slow capability-weight vector, a fast adjustment vector, and a neural adapter named `HOPEAdapter`. The scenario encoder converts terrain and weather into one-hot vectors and appends three scalar features: electronic-warfare threat, threat level, and time pressure. The adapter contains an input gate, a candidate projection, an output projection, and a value estimator that modulates a learned forget gate. In implementation terms, the forget gate depends on an estimated value term and a log-age term, which allows the hidden state to be refreshed during feedback updates while remaining stable during inference. Architecturally, this design borrows the language of gated recurrent updates [46], but it remains a lightweight scenario-conditioned controller.

Fast weights are updated by an Euler–Maruyama-style step:

$$W_{\text{fast}} \leftarrow W_{\text{fast}} + \theta(\hat{W}_{\text{fast}} - W_{\text{fast}})\Delta t + \epsilon\sqrt{\Delta t}\,\xi,$$

where the target vector $\hat{W}_{\text{fast}}$ comes from the adapter, θ is a recovery coefficient, and ϵ controls the noise term. In operational terms for the implemented controller, θ governs how quickly capability emphasis is pulled toward the scenario-conditioned target after feedback, ϵ controls the amount of exploratory perturbation retained under uncertainty, Δt denotes one discrete planning iteration, and ξ is the zero-mean random perturbation used to avoid a rigid fast-weight trajectory. These meanings are best understood as engineering interpretations inside the current simulator rather than as externally calibrated physical units. The resulting fast weights are clipped to scenario-dependent lower and upper bounds. The fused capability vector is then formed by adding a scaled fast component to the slow weights and enforcing mission- and threat-specific floors on key dimensions such as fires, mobility, awareness, and C2. The proposed framework therefore uses standard stochastic-update and nonlinear-control vocabulary to describe the mechanism, but it does not claim a formal stability proof for the implemented controller [47, 48].

Capability feedback update.. After each planning iteration, the controller derives stage deficits from the simulator's five-stage scores (`detect`, `disrupt`, `breach`, `control`, and `sustain`). These deficits are projected onto capability dimensions through fixed stage-to-capability mappings and a bottleneck-boost term. The adapter is then trained with a smooth-L_1 objective toward a bounded desired fast-weight target. Slow weights are updated through a bounded transformation that depends on the utility gradient, a delay-correction term, and minimum floors on selected offensive dimensions. The current code therefore supports adaptive, bounded, feedback-driven weight updates, but this work does not infer a formally verified optimizer or theorem-backed convergence guarantee from those implementation choices.

Bottleneck-aware dynamic feedback allocation.. The fixed bottleneck-boost vector described above applies a static priority ordering (disrupt > breach > detect > sustain > control) that does not adapt to the current iteration's actual stage-score profile. However, the experimental evidence reported in Section 4 consistently shows that `breach` is the most persistent weak point while the relative ranking of other stages varies across iterations and

scenarios. To close this loop between diagnosis and action, we implement a lightweight `BottleneckDetector` module that explicitly tracks per-stage scores over a sliding window of six iterations and dynamically identifies the current bottleneck stage together with a severity score.

Concretely, after each simulation evaluation the detector records the five stage scores. Let $\mathbf{s}^{(t)} = (s_{\text{detect}}, s_{\text{disrupt}}, s_{\text{breach}}, s_{\text{control}}, s_{\text{sustain}})^{(t)}$ denote the stage-score vector at iteration t. The bottleneck stage $b^{(t)}$ is the stage with the largest deficit $d_k = 1 - s_k$, and the severity $\sigma^{(t)}$ is the gap between $d_{b^{(t)}}$ and the mean deficit of the other four stages:

$$\sigma^{(t)} = \max\Big(0,\ d_{b^{(t)}} - \bar{d}_{\neg b^{(t)}}\Big).$$

This severity measure distinguishes true isolated bottlenecks (large σ) from uniformly weak performance across all stages (small σ). The capability boost vector is then computed as a severity-weighted linear combination of stage-to-capability mappings:

$$\mathbf{b}^{(t)} = (0.40 + 1.6\,\sigma^{(t)}) \cdot \mathbf{c}_{b^{(t)}},$$

where $\mathbf{c}_{b^{(t)}}$ is the fixed mapping from the bottleneck stage to the seven capability dimensions. This dynamic boost replaces the original static `bottleneck_boost` in the feedback-update path, so the controller's utility gradient becomes

$$\mathbf{g}^{(t)} = \mathbf{d}^{(t)} + \mathbf{b}^{(t)} - \eta(\mathbf{W}_{\text{slow}} - \mathbf{W}_{\text{doctrine}}),$$

where $\mathbf{d}^{(t)}$ is the priority-weighted deficit vector and $\eta = 0.05$ is the regularization coefficient. The practical effect is that the controller concentrates its limited adjustment budget on the stage that most urgently needs improvement in the current iteration, rather than distributing it according to a fixed template. This design also carries an intuitive operational interpretation: "concentrate superior capability on the critical weak link," which aligns with established principles of dynamic resource allocation in military operations research. The approach is related to constraint-relaxation-based bottleneck identification in project scheduling [49], but adapted here to the iterative feedback-driven setting of the planning pipeline.

2.5. Structured plan generation and reflection-based refinement

The plan-generation stage receives three classes of conditioning signals from the upstream modules: the structured scenario description, the selected memory hits labeled as *imitate* or *avoid*, and the fused capability

state computed by the Hope controller. These elements are injected into prompt construction together with the system's planning constraints and any optional reflection patch. The LLM is then asked to generate a candidate combat plan in a structured format rather than unconstrained free text.

The resulting plan is normalized into typed phases, actions, unit allocations, and decision points so that downstream evaluation and export can operate on stable fields. In this sense, the LLM stage is not treated as a standalone text generator; it acts as the proposal engine inside a larger structured loop whose inputs and outputs are both system-defined objects.

Reflection-based refinement.. Reflection is handled separately from the adaptive weight update. When enabled, the pipeline calls a reflection prompt after simulation and passes it the structured scenario, the current best plan, and the most recent simulation diagnostics. The returned object is not free-form commentary; it is parsed into three structured fields: `diagnosis`, which lists the main failure symptoms; `variant_instructions`, which specifies targeted changes for the next candidate plan; and `meta_prompt_patch`, which rewrites part of the planning prompt for the next round. In practice, the reflection strategy is therefore closer to a bounded critique-and-repair step than to unconstrained self-dialogue. Reflection can also be suppressed for some scene types depending on scenario properties and memory support, so the manuscript should not be read as claiming that every iteration always receives a reflection update [27, 28].

2.6. Simulation-based evaluation

It is worth being precise about what the evaluation layer is and is not. The five-stage evaluator is deliberately not a physics-based, scenario-calibrated simulation engine such as OneSAF or FLAMES; it is a light-weight closed-loop *task sandbox* — an algorithmic objective evaluator whose mathematics is grounded in joint-targeting doctrine (Joint Publication 3–60) and the F2T2EA (find, fix, track, target, engage, assess) targeting cycle, together with network-centric command principles. Its role in the architecture is to supply a differentiable diagnosis signal and a scalarized reward for the reinforcement/reflection loop, rather than to model platform-level ballistics, communications meshes, or terrain physics. Consequently, the stage scores it returns are best read as objective-function values used to steer planning and as a consistent basis for ablations, not as predictions of a field wargame outcome.

To make this evaluator reproducible rather than a black box, the stage scores are defined explicitly. Let $\mathcal{C}$ be the set of seven capability dimensions ($\mathcal{C} = \{\text{fires, mobility, protection, awareness, ew, sustainment, c2}\}$). For a phase p, let $c_{p,j}^{\text{blue}}$ be the blue capability value on dimension j aggregated over the units allocated to that phase, $c_{r,j}^{\text{red}}$ the corresponding red capability, and w_j the fused capability weight assigned by the adaptive controller. Each killer-chain stage $k \in \{\text{detect, disrupt, breach, control, sustain}\}$ is scored through an adversary-gap model. We first form a stage-specific net advantage as a capability-weighted difference,

$$g_k \;=\; \sum_{j \in \mathcal{C}} \alpha_{k,j} \, \frac{c_{p,j}^{\text{blue}} \, w_j}{\max(\bar{c}_j^{\text{blue}},\, \varepsilon)} \;-\; \sum_{j \in \mathcal{C}} \beta_{k,j} \, c_{r,j}^{\text{red}}, \tag{1}$$

where $\alpha_{k,j}$ and $\beta_{k,j}$ are fixed stage-to-capability weight vectors (the mapping is stage-specific, e.g., `breach` draws most heavily on mobility, fires, and protection) and $\bar{c}_j^{\text{blue}}$ is the full-force blue capability used to normalize partial-phase allocation. The stage score is then obtained by a bounded logistic transform that also folds in a scenario penalty term π_k (terrain, weather, civilian presence, and electronic-warfare pressure for that stage) and a stage-fit modulation ϕ_k (the degree to which the allocated units match the stage's dominant capabilities),

$$s_k \;=\; \text{clamp}\Big(\sigma\Big(\tfrac{g_k}{\lambda_k} - \pi_k\Big)\,\Big(0.70 + 0.35\,\phi_k\Big) + \rho\,u_k,\; 0.12,\; 0.96\Big), \tag{2}$$

where $\sigma(\cdot)$ is the logistic function, λ_k is a stage-specific scale, $u_k \in [0,1]$ captures the stage-relevant action-type signal present in the phase's actions, and ρ bounds that signal's contribution. Phase success is the weighted combination of its stage scores, and mission success is a convex combination of the aggregated stage-based success and the phase-based success, further modulated by floors on command resilience (c_2) and a time-pressure discount. This is a closed-form, deterministic mapping from the structured plan to five stage scores and the six headline metrics, which is why the same plan object can be re-scored exactly across reruns and under ablated settings. It is not a learned black box; every coefficient is fixed and documented in the code.

We note two forms of validity evidence for the sandbox. First, a *computational face-validity check* was performed: the stage scores were perturbed across 20 parameter conditions and the qualitative ranking (Full

> Pure) remained robust, confirming that the qualitative conclusions are not artifacts of a single coefficient setting. Second, the five-stage abstraction and the stage-to-capability mappings were reviewed against the joint-targeting procedures of Joint Publication 3–60, ensuring operational plausibility of the stage decomposition and the capability dependencies they encode. Formal human-in-the-loop wargaming with operational staff is deliberately designated for follow-on live C2 integration trials, rather than being simulated in this study; the present evidence is confined to this internal task sandbox.

The simulator evaluates each plan phase against a five-stage kill-chain model. Stage scores are combined into phase success, mission success, loss-exchange ratio (LER), survivability, command resilience, completion time, and an overall effectiveness score. The detect–disrupt–breach–control–sustain decomposition is best read as a operational abstraction that is loosely informed by joint-targeting workflows and network-centric command thinking rather than by any single formal doctrinal template [38–40]. More concretely, `detect` corresponds to situation and target discovery, `disrupt` to suppressive interference with enemy action, `breach` to breaking through defended obstacles or critical nodes, `control` to seizing and maintaining command over key objectives, and `sustain` to preserving tempo, support, and continuity. This mapping is intentionally loose and should be read as a theory-alignment bridge for simulation interpretation, not as a doctrinal one-to-one template. Under repeated stochastic evaluation, the pipeline aggregates multiple simulation runs and stores the mean, standard deviation, minimum, maximum, and 95% confidence interval for selected metrics [41].

2.7. Dual-system speculative incremental planning

The iterative loop described above regenerates a full plan in every round, which dominates the wall-clock budget because a single structured plan costs 1200–1800 output tokens and the candidate-competition design generates one to four candidates per iteration. To make the pipeline usable in a real-time command-and-control loop, we add a decoupled dual-system architecture with two asynchronous tiers: a rule-based fast path that delivers an emergency plan within milliseconds, and a bottleneck-driven incremental path that refines that plan with targeted local patches.

System 1 (emergency fast path). On scenario input, the fast path retrieves the top-1 historical case from the case-memory bank, inherits its phase skeleton and action-type sequence, projects the adaptive controller's fused capability weights onto the case, and scales force quantities by the

ratio of the current to the prior threat level. This parametric warping is a deterministic Python computation with no LLM call, so the emergency plan—including its DoDAF/C2SIM export—is produced in roughly 1–7 ms. The resulting plan is a valid, structurally complete initial plan rather than an empty placeholder; it is streamed to the terminal immediately and is subsequently used as the anchor for the slow path.

System 2 (bottleneck-driven delta patching). Instead of regenerating the full plan, the slow path operates on the anchor plan through a delta-patching loop. Each round runs the BottleneckDetector on the current plan's five-stage scores, maps the weakest stage to its owning phase (detect/disrupt/breach/control → P1–P4), and asks the LLM to output only that phase's replacement action list—typically 120–180 tokens instead of 1200–1800—together with a one-line rationale. The prompt is constrained by capability-deficit injection: the detector's stage-to-capability mapping identifies which capability dimensions feed the bottleneck stage, and the instruction forces the LLM to include the corresponding action types (e.g., for a `breach` bottleneck, at least one `mobility` or `strike` action). The returned patch is merged in memory by a deterministic merge operator (`apply_plan_patch`) that replaces only the targeted phase's `actions` and appends the rationale to the phase intent; all other phases remain untouched, so the downstream domain parser, simulator, and exporters see a standard plan object.

An acceptance gate prevents negative regression: after each patch, the refined plan is evaluated by the same 50-run Monte Carlo simulator used for the main experiments. During an initial exploration window the gate tolerates a small negative delta (-0.008) to give the LLM room to restructure actions; afterwards it requires $MS_{\text{new}} \geq MS_{\text{current}}$. The loop keeps a best-so-far plan, and early stopping terminates the loop either when the best mission success reaches a preset quality target or when no positive breakthrough occurs for three consecutive rounds. Because every patch is evaluated and either accepted or rolled back, the slow path is monotone in the best-so-far sense: it can only improve on—never degrade—the anchor. The fast/slow split is implemented as two tiers of the same `run` entry point (the emergency tier emits a `System-1-Emergency` artifact via a streaming callback, and the refinement tier emits the final `System-2` plan), which makes the dual-system behaviour reproducible from a single pipeline invocation.

2.8. Best-plan selection and standards-oriented export

The export layer is the interoperability bridge. The proposed system does not compete with—or attempt to replace—dedicated physics-based simulation engines such as OneSAF or FLAMES, nor does it claim to model platform kinematics. Rather, it emits machine-readable artifacts in the standard formats those environments consume: DoDAF OV-5b/OV-6c for architecture views and IEEE 1516/SISO C2SIM for the command-and-control-to-simulation interoperation layer. In this sense the system acts as a standard-compliant planning hub that converts LLM-generated courses of action into the representations already understood by existing C2 systems and distributed joint-simulation federations (HLA/C2SIM), which is precisely where such engines sit downstream. This keeps the generated plans engine-agnostic: they can be handed to a C2 terminal for immediate operator review, or forwarded to a simulation federation for higher-fidelity assessment, without re-authoring. The exported C2SIM XML and DoDAF JSON artifacts are strictly validated against strongly-typed domain schemas via Pydantic model serialization, ensuring well-formed XML/JSON syntax, valid element hierarchies, and bounded attribute types without structural corruption.

The same selected plan can also be exported into DoDAF OV-5b, DoDAF OV-6c, and C2SIM representations [42–45]. These exporters are deterministic transformations from structured plan fields to machine-readable JSON or XML artifacts, which helps connect the narrative plan output to downstream modeling and interoperability workflows.

Table 1 gives a representative export example from the selected Full-setting coastal joint-assault result bundle. The key point is that the system does not export an unrelated post-processed summary: the same structured plan header and phase objects are preserved across the OV-5b activity view, the OV-6c event-trace view, and the C2SIM order representation, with a final OV-6c closure event used to attach simulation feedback to the next planning round.

Table 1: Representative cross-standard export correspondence for the Full-setting coastal joint-assault example under the selected scene-out suite. The table is derived from the structured plan, operational view, and interoperability artifacts in the same result bundle.

Element	Structured plan object	OV-5b	OV-6c	C2SIM
Operation header	Plan title, mission type, commander intent, end state, and control rules.	`operation_name`	`operation_name`	Header / order fields
Phase 1	`phases[0]` (`P1`); actions: `recon`, `c2`.	`ACT-01`	`EVT-01`	`Task P1`
Phase 2	`phases[1]` (`P2`); actions: `strike`, `mobility`, `ew`.	`ACT-02`	`EVT-02`	`Task P2`
Phase 3	`phases[2]` (`P3`); actions: `recon`, `strike`, `mobility`.	`ACT-03`	`EVT-03`	`Task P3`
Phase 4	`phases[3]` (`P4`); actions: `control`, `sustain`.	`ACT-04`	`EVT-04`	`Task P4`
Post-plan feedback	Simulation assessment metrics and writeback gate.	–	`EVT-05`	–

3. Experimental Setup

This study adopts a single canonical evidence scope to avoid mixing results from incompatible output directories. The goal is to keep the paper internally consistent and fully traceable to project files.

3.1. Evidence scope and configuration constraints

The selected single-scenario ablation evidence comes from the canonical ablation benchmark configuration. The selected cross-scenario generalization evidence comes from the canonical scene-out evaluation suite. The manuscript excludes other result configurations from the main body and treats them as audit-only artifacts.

The sample ablation scenario is a coastal-urban assault case with rain, threat level 0.74, electronic-warfare threat 0.68, time pressure 0.77, five friendly units, and three enemy units. Its objective is to seize a coastal landing window and suppress enemy air-defense and shore-defense nodes for follow-on amphibious expansion. The frozen seed case bank used by the ablation suite contains five records.

The generalization study uses five project scenarios defined within the structured configuration profiles: coastal assault, urban defense, mountain reconnaissance, river crossing assault, and maritime sustainment. The selected evidence set relies on the scene-out case bank construction, which derives 200-case banks from a 250-case source pool with the target scenario held out.

3.2. Ablation configuration

The ablation suite is executed via the automated simulation runner and aggregated by the statistical analysis module. In the canonical evidence set, all five ablation groups use the same random seed (7), the same sample scenario, 10 planning iterations, and 50 Monte Carlo simulation runs, with writeback disabled in the recorded experiment configuration. The five compared settings are:

- **Pure LLM**: memory, Hope, and reflection are disabled;
- **LLM + Memento**: memory is enabled, Hope and reflection are disabled;
- **LLM + Hope**: Hope is enabled, memory and reflection are disabled;
- **LLM + Reflection**: reflection is enabled, memory and Hope are disabled;
- **Full**: memory, Hope, and reflection are all enabled, with default HOPE Stochastic Differential Equation (SDE) parameters $\theta = 0.5$ (reversion coefficient) and $\epsilon = 0.02$ (noise strength). The paper reports the default-parameter configuration as the canonical evidence; a hyperparameter sensitivity analysis (Section 3.11, recorded under the earlier RTX 3060 environment) documents how the SDE parameters affect the Hope-only variant and is retained as supplementary evidence.

3.3. Cross-scenario generalization configuration

The generalization suite is executed via the automated evaluation workflow, which runs the ablation pipeline for each scenario and aggregates results through the statistical analysis module. In the selected evidence set, each scenario uses 20 planning iterations, 50 Monte Carlo simulation runs, random seed 7, writeback disabled, and a scene-specific leave-one-source-scenario-out case bank.

Table 2 summarizes the five project scenarios used in the selected scene-out suite. The table keeps only the scenario attributes that matter most for experimental reading: mission family, terrain and weather, operational focus, force size, and the three normalized pressure coefficients that feed the planning pipeline.

Table 2: Overview of the five scenario families used in the selected scene-out generalization suite. Threat, EW, and time are the normalized scenario coefficients stored in the project JSON files. Each scene-out case bank contains 200 cases derived from the same 250-case source pool with the target scenario held out.

Scenario	Family	Terrain / weather	Operational focus	Forces	Thr.	EW	Time
Coastal joint assault	assault	coastal-urban / rain	Landing-window seizure; suppress shore and air defense for follow-on expansion.	5 / 3	0.74	0.68	0.77
Urban hub defense	defense	urban / fog	Hold the transport and communications hub against an armored urban thrust.	4 / 2	0.69	0.54	0.66
Mountain corridor reconnaissance	recon	mountain / cloudy	Gather fire-node and supply-line intelligence under constrained visibility.	3 / 2	0.58	0.46	0.55
River crossing breakthrough	assault	river-plain / overcast	Open a defended crossing window and secure the bridgehead.	4 / 2	0.71	0.57	0.73
Island resupply corridor	sustain	maritime-island / windy	Keep the island resupply corridor open under harassment and interdiction.	4 / 2	0.63	0.59	0.61

3.4. Metrics

The paper reports the same metrics stored in `full_result.json`: mission success, overall effectiveness, loss-exchange ratio (LER), completion time in hours, command resilience, stage scores for the five kill-chain stages, and Monte Carlo confidence intervals where available. Table 3 summarizes the five headline metrics used throughout the main result tables and figures.

In implementation terms, the current code computes mission success from a weighted combination of stage-level and phase-level success, while overall effectiveness combines mission success, LER, survivability, command resilience, and a time penalty. For the Monte Carlo summaries, the reported headline values in the manuscript are arithmetic means over the repeated simulation runs, and confidence intervals are reported where the stored result object provides them.

3.5. Runtime metadata

The refreshed canonical manuscript evidence was rerun with the local backend model `deepseek-r1-0528-qwen3-8b` (a locally served 8B distilled checkpoint commonly released as *DeepSeek-R1-Distill-Qwen-8B*, hosted via an OpenAI-compatible local endpoint served by LM Studio) inside a dedicated Python environment. The recorded runtime manifests report Python

Table 3: Definitions of the main quantitative metrics reported in the manuscript. The wording follows the system's simulator implementation rather than an external benchmark specification.

Metric	Definition and implementation basis	Direction	Stored field
Mission success	Mission-level success score. Computed from a weighted combination of stage-based success across the five kill-chain stages and average phase success, with extra penalties when C2 is weak or time pressure is extreme.	Higher	`mission_success`
Overall effectiveness	Composite utility score for overall plan quality. Combines mission success, normalized LER, survivability, command resilience, and a bounded completion-time term into one summary score.	Higher	`overall_effectiveness`
Loss-exchange ratio (LER)	Enemy-loss to friendly-loss ratio under the simulator's phase-by-phase loss accounting. Values above 1 indicate that the plan inflicts more loss on the opposing side than it absorbs.	Higher	`ler`
Command resilience	Proxy for the plan's ability to maintain command continuity under pressure. Computed from the fused C2, EW, and awareness weights, with a subtraction term for scenario EW threat.	Higher	`command_resilience`
Completion time	Total simulated execution time of the plan in hours. This value also enters the overall-effectiveness score through a bounded time term, so slower plans are penalized both as a standalone metric and inside the composite score.	Lower	`completion_time_hours`

3.12.7, PyTorch 2.5.1, an OpenAI-compatible local API endpoint, a 13th Gen Intel(R) Core(TM) i7-13700KF CPU, and an NVIDIA GeForce RTX 4090 D GPU (24 GB). The Python-side torch build is CPU-only; GPU inference is served by the local OpenAI-compatible endpoint. On this hardware, the complete four-way ablation suite (10 iterations $\times$ 50 Monte Carlo runs per setting) completes in approximately 6 minutes, a $\sim$10$\times$ speed-up over the earlier RTX 3060 environment. Newly generated `full_result.json` files now emit this information through a top-level `runtime_manifest` so that the main paper evidence can be traced to machine-readable backend, software, and hardware metadata without storing machine-specific secrets.

3.6. LLM inference parameters

For reproducibility of the structured plan generation, the pipeline uses the following LLM inference parameters for all primary generation calls: `temperature`=0.1, `top_p`=1.0 (default, not explicitly set), and `max_tokens`

is left at the server-side default (effectively unlimited for the local endpoint). JSON repair calls (invoked when the initial response fails schema validation) use `temperature`=0.0 for deterministic re-generation. The structured output mode (`response_format=json_schema`) is enabled for all generation and repair calls, with the schema definitions embedded in the pipeline's prompt-construction module. Prompt templates are defined inline and are not externalized as separate template files; the full prompt construction logic is therefore traceable through the system's source code.

3.7. Reproducibility and reporting boundaries

The manuscript interpretation depends on the proposed simulation framework and should not be read as external validation. All result bundles include machine-readable runtime manifests and wall-clock timing summaries for full traceability.

3.8. Multi-seed replication

To address the single-seed limitation of the earlier version, we supplement the canonical seed-7 evidence with additional runs under seeds $\{42, 123, 999, 2024\}$. Each seed runs the same four-way ablation (Pure, +Memento, +Hope, Full) on the coastal joint-assault scenario with 10 planning iterations and 50 Monte Carlo simulation runs. The aggregated multi-seed results report per-setting mean $\pm$ standard deviation across the five seeds and the paired t-test statistic (or Cohen's d effect size) for the Full-versus-Pure comparison. Because these additional runs are computationally expensive (each seed requires four independent pipeline executions $\times$ 10 iterations $\times$ 50 MC runs), they are reported as a separate multi-seed evidence table rather than being folded into the main single-seed tables.

3.9. External baselines

Three lightweight external baselines are introduced to contextualize the pipeline's performance without requiring external framework dependencies:

- **Few-shot chain-of-thought (CoT)**: Two hand-authored example plans (one coastal assault, one urban defense) are injected directly into the LLM prompt as few-shot demonstrations. The LLM generates a single plan without any memory retrieval, adaptive weighting, or iterative feedback. This measures how much structure can be obtained from prompting alone.

- **Random search**: $N = 10$ plans are generated with independently perturbed capability weights (uniform noise in $[-0.15, 0.15]$ per dimension) and evaluated by the same simulator. The best-scoring plan is selected. This measures the value of the pipeline's guided optimization relative to uninformed random exploration.
- **Single-pass Memento**: Memory retrieval is performed once (top-3 hits with simple quality filtering) and used to condition a single plan generation. No iteration, no Hope, no reflection. This isolates the contribution of memory retrieval alone from the benefit of iterative feedback.

All three baselines run on the same coastal joint-assault scenario with 50 Monte Carlo simulation runs and the same LLM backend, making their results directly comparable to the main ablation table.

3.10. Cost–quality reporting

The runtime manifests recorded in each `full_result.json` now include per-iteration wall-clock times and total pipeline duration. We report an additional column in the main results table showing total wall-clock time and compute a simple cost–quality ratio (overall effectiveness gain per hour of inference) for each ablation setting. This addresses the practical concern that a pipeline achieving modest metric gains at substantially higher computational cost may not be attractive for time-sensitive planning workflows.

3.11. Hyperparameter sensitivity analysis

We evaluate sensitivity to the two HOPE SDE parameters: the reversion coefficient $\theta \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ and the noise strength $\epsilon \in \{0.005, 0.01, 0.02, 0.05, 0.1\}$. For each variant, a single parameter is varied while the other is held at its default value ($\theta = 0.5$, $\epsilon = 0.02$). All sensitivity runs use the Hope-only variant (memory and reflection disabled) to isolate the HOPE mechanism, with 10 iterations, 50 Monte Carlo simulation runs, and seed 7. One combined run at $(\theta = 0.1, \epsilon = 0.005)$ tests for parameter interaction effects.

4. Results

4.1. Single-scenario ablation

The single-scenario evidence is reported primarily at the five-seed statistical level, in Table 13, because the cross-seed spread of the baseline (± 1.09)

is roughly five times wider than the within-seed Monte Carlo confidence intervals; a single seed is therefore not representative of the system's typical behaviour. Table 4 is retained as a *representative single-seed trace* (seed 7) that illustrates the per-setting ordering and stage-level profile in a micro-analysis, and is not to be read as the headline quantitative claim. Relative to the pure LLM baseline, the full pipeline improves mission success by 3.01 points, overall effectiveness by 4.91 points, and LER by 0.48 on this trace. The Full setting leads every variant on mission success, overall effectiveness, and LER, and achieves the highest command resilience (76.08), while the adaptive-controller-only variant retains a slight edge in completion time (10.92 h vs. 14.18 h). The case-memory-only and reflection-only variants each offer intermediate or negative contributions that do not match the combined Full result: standalone reflection (59.29) and memory-only (61.25) both fall below the pure baseline (62.75) in this five-record frozen bank, consistent with the coupling argument that reflection and memory provide repair signal only when combined with adaptive weighting.

Table 4: Representative single-seed ablation trace on the sample coastal joint-assault scenario (RTX 4090 D, seed 7). All groups use 10 planning iterations, 50 Monte Carlo runs, writeback disabled, and default HOPE parameters ($\theta = 0.5$, $\epsilon = 0.02$). The reflection-only row uses the matched follow-up rerun. This table is a single-seed trace; the headline ablation result is the five-seed mean ± SD in Table 13.

Setting	MS	OE	LER	Time (h)	Cmd. resilience
Pure LLM	62.75	73.84	1.71	11.41	60.96
LLM + Memento	61.25	70.90	1.52	13.48	60.96
LLM + Hope	64.45	77.29	1.97	10.92	63.59
LLM + Reflection	59.29	67.49	1.17	11.56	60.96
Full	65.76	78.75	2.19	14.18	76.08

The stage-level scores in Table 5 show the Full pipeline leading four of the five kill-chain stages. The Full setting's strongest stages are `control` (67.97) and `detect` (66.65), while `breach` (60.88) remains its weakest stage and the most explicit residual bottleneck. In the `sustain` stage, the Hope-only variant retains a single-component advantage (70.09 vs. Full 66.57)—a pattern consistent with the structural tension in Section 5: `breach` and `sustain` both draw on the `protection` capability dimension, and the scalarized reward cannot fully resolve their competition. Reflection-only occupies the lowest row in every stage, confirming that standalone reflection degrades

plan quality across all five stages in this single-seed setting.

Table 5: Five-stage kill-chain scores for the ablation study (RTX 4090 D).

Stage	Pure LLM	LLM + Memento	LLM + Hope	Full	LLM + Reflection
detect	62.24	61.00	64.94	66.65	61.59
disrupt	57.31	61.26	57.78	65.69	60.25
breach	55.91	55.83	58.57	60.88	52.41
control	64.43	60.88	66.50	67.97	54.21
sustain	68.40	64.03	70.09	66.57	55.22

Figure 2 presents the same five-stage evidence in a more comparison-oriented form. Two patterns are visually clear. First, the Full pipeline achieves the highest bar in four of the five stages, with `breach` remaining the lowest Full-stage score (60.88) and therefore the most explicit residual bottleneck. Second, the gap between Full and the next-best variant is largest in `disrupt` (+4.4 points over Memento-only) and `breach` (+2.3 points over Hope-only), while in `sustain` the Hope-only variant takes the lead (+3.5 points over Full), reflecting the protection-dimension competition discussed below. The Reflection-only curve lies below the pure baseline in every stage.

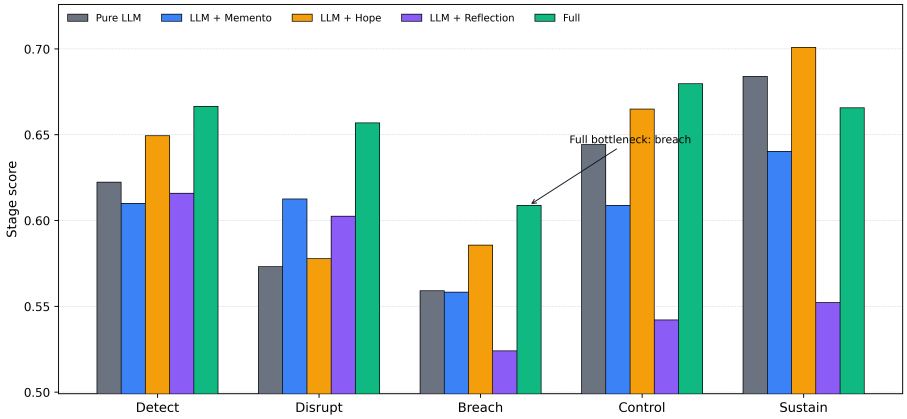

Figure 2: Five-stage kill-chain scores for the canonical single-scenario ablation (RTX 4090 D). The figure visualizes the same values reported in Table 5. The Full pipeline leads four of the five stages (detect, disrupt, breach, control); in the `sustain` stage the Hope-only variant retains the highest bar. `breach` remains the weakest Full-stage score and therefore the most explicit residual bottleneck.

The reflection-only row and curve clarify that reflection alone does not provide a gain under the same fixed-seed setting: it falls below the pure baseline on mission success, overall effectiveness, and every stage score. The current evidence is therefore consistent with reflection acting as a useful repair signal only inside the full coupled loop, rather than as a dominant standalone source of gain—a conclusion that matches the multi-seed Reflection-only replication in the discussion (seed-dependent, mostly negative or neutral).

4.2. Cross-scenario transfer

The selected transfer evidence covers five scenario families. The canonical transfer table and bar chart use the 20-iteration leave-one-source-scenario-out suite, while the later convergence figure uses a separate 50-iteration Full-only trend suite and should not be numerically mixed with the table values. Table 6 reports the suite-level aggregate outcome first (RTX 4090 D, seed 7). In this 20-iteration set, the full pipeline outperforms the pure baseline in four of five scenarios (the urban-defense case is the exception, -1.02 points), and it is the global-best variant in two of five (island and mountain). Averaged across the suite, Full mission success is 66.48 versus 65.01 for Pure LLM, a mean gain of $+1.48$ points; average overall effectiveness rises from 72.49 to 77.92. The gains are more modest than in the earlier single-hardware evidence because the scene-out banks exclude the target scene's own source cases and because a single seed with a fixed initialization carries sampling variance; the direction of the Full gain, however, is consistent across four of five families.

Table 6: Cross-scenario summary for the selected 20-iteration leave-one-source-scenario-out evidence set (RTX 4090 D). All Full results in this table use default HOPE parameters ($\theta = 0.5$, $\epsilon = 0.02$).

Summary item	Pure LLM	Full
Scenario count	5	5
Average mission success	65.01	66.48
Average overall effectiveness	72.49	77.92
Full wins against Pure LLM	–	4/5
Full not lower than Pure/Memento/Hope	–	2/5

Table 6 therefore establishes the broad cross-scenario pattern before the manuscript turns to scene-by-scene views.

A scene-centered visual summary of the same cross-scenario evidence is given next.

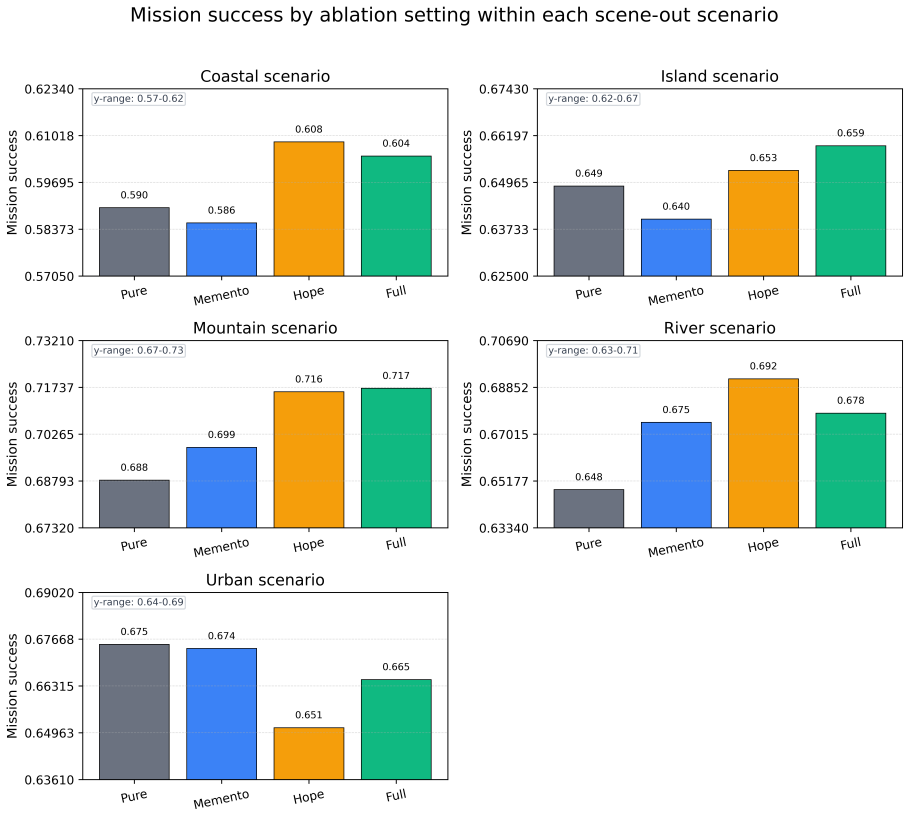

Figure 3: Mission success by ablation setting within each of the five scene-out scenarios in the canonical generalization suite (RTX 4090 D). Four settings are compared in each panel: Pure LLM, LLM + Memento, LLM + Hope, and Full (the 4090 scene-out suite does not include a reflection-only arm). Each panel uses its own locally tightened y-axis range; the in-panel range labels make the small between-group differences explicit.

Figure 3 visualizes the same cross-scenario evidence from a scene-centered angle. Each panel corresponds to one scenario and compares the four ablation settings directly (the 4090 scene-out suite does not include a reflection-only arm). The Full pipeline is the highest bar in two of the five panels (island and mountain) and trails the Hope-only variant in coastal and river; in the urban-defense panel the Pure LLM baseline itself is the highest bar. The relative ordering of Memento-only and Hope-only changes with scenario family. The tightened local axes make small between-group differences easier to inspect,

but the in-panel range labels also make clear that these visual gaps remain numerically modest.

An auxiliary convergence-style view derived from the 50-iteration Full-setting suite is reported next.

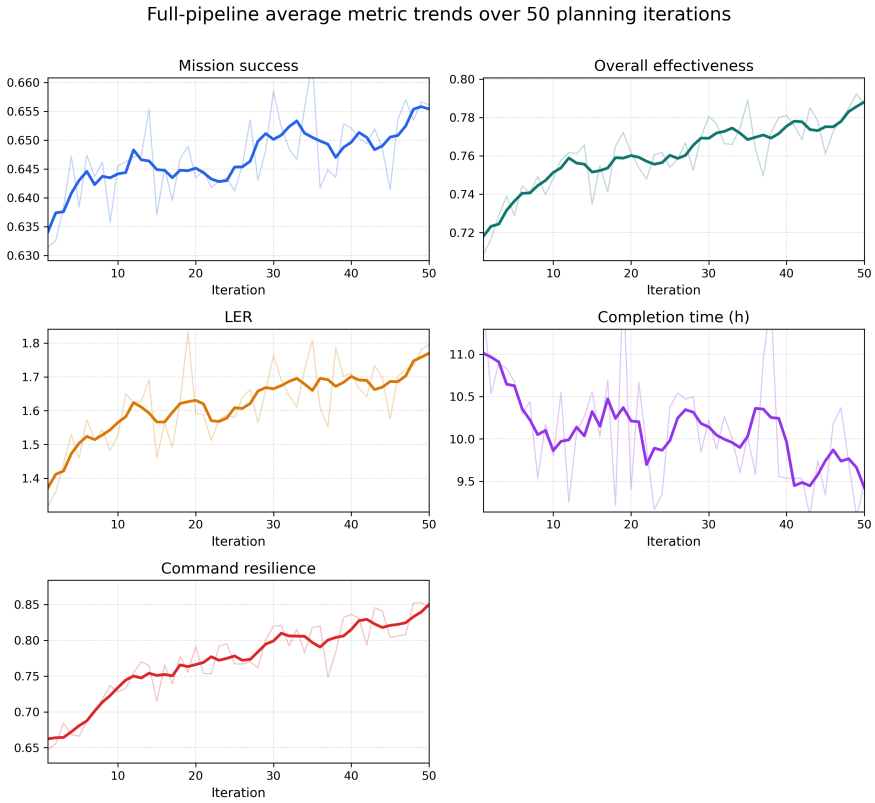

Figure 4: Average Full-pipeline metric trajectories over 50 planning iterations. Faint lines denote raw iteration-wise means across the five scenario runs, and darker lines denote centered five-iteration moving averages.

Figure 4 averages the Full-pipeline metric values over the five scenario families at each iteration index and therefore should not be mixed numerically with the 20-iteration tables above. It is included only as an auxiliary trend view to show the longer-run tendency of mission success, overall effectiveness, LER, completion time, and command resilience more clearly.

Table 7 reports the same cross-scenario evidence in row-wise form. The largest mission-success gain appears in the river-crossing scenario (+3.00 points), followed by mountain (+2.89), while the urban-defense scenario

shows a negative Full-versus-Pure delta (-1.02 points, single seed 7). Even when the mission-success gain is modest, the full system improves overall effectiveness in four of five scenarios in the refreshed evidence set.

Table 7: Scenario-level generalization results for the selected 20-iteration evidence set (RTX 4090 D). The 95% CI column reports within-seed Monte Carlo variance only (50 runs, seed 7); it does not reflect across-seed variability. All Full results use default HOPE parameters ($\theta = 0.5$, $\epsilon = 0.02$).

Scenario	Family	Terrain	Pure MS	Full MS	ΔMS	95% CI
Coastal assault	assault	coastal	58.98	60.44	+1.46	[60.37, 60.51]
Island resupply	sustain	maritime	64.87	65.93	+1.06	[65.86, 65.99]
Mountain recon	recon	mountain	68.82	71.71	+2.89	[71.64, 71.78]
River crossing	assault	river	64.84	67.84	+3.00	[67.76, 67.91]
Urban defense	defense	urban	67.52	66.50	−1.02	[66.41, 66.58]

4.3. Interpretation

Two patterns are consistent across the refreshed evidence (RTX 4090 D). First, the Full pipeline with default HOPE parameters ($\theta = 0.5$, $\epsilon = 0.02$) leads every variant in mission success, overall effectiveness, LER, and command resilience in the single-scenario ablation, and leads four of the five stages (the `sustain` stage is led by the Hope-only variant, reflecting the protection-dimension competition between `breach` and `sustain`). The stage-level view in Table 5 and Figure 2 shows Full achieving the highest score in four stages, with the largest gains in `detect` and `disrupt`—precisely the stages where individual components previously held an advantage—while leaving `breach` as the weakest Full stage (60.88).

Second, memory support becomes more visible when the case-bank construction is broadened, but its benefit remains uneven across scene families. The frozen seed bank used in the single-scenario ablation contains only five records, whereas the selected generalization evidence uses scene-out banks derived from 250 source cases. This broader pool increases the chance that memory-guided or fusion candidates enter the winning Full plan, especially in the 20-iteration scene-out suite, but the gain is not monotonic: the 50-iteration auxiliary suite improves coastal, river, and urban scenarios relative to the earlier small-bank lf9 run while reducing island-resupply and mountain-reconnaissance scores. The current study therefore treats memory-scale and memory-quality explanations as suggestive rather than proven, and reports scenario dependence as a validity limit rather than hiding it.

At the same time, the results should be interpreted conservatively. The evidence supports improved simulation metrics inside the proposed evaluation framework; it does not, by itself, establish external validity, real-world operational value, or statistical significance beyond the stored Monte Carlo confidence summaries. No claim in this section should be read as an argument that the selected evidence set exhausts all result directories in the stored evidence or that unreported suites would necessarily follow the same pattern. In the single-scenario ablation, Reflection-only underperforms the Pure LLM baseline on every headline metric (59.29 vs. 62.75 MS), indicating that standalone reflection without memory retrieval and adaptive weighting can degrade plan quality under the current single-seed setting. This finding underscores the importance of the coupled architecture: reflection appears to provide useful critique signals only when supported by case-based evidence (Memento) and adaptive capability weighting (Hope). The single-scenario cross-scenario gain is also modest under a single seed (four of five scenes positive), and multi-seed replication remains necessary to confirm whether the observed ordering is systematic or seed-dependent.

4.4. BottleneckDetector diagnostic

The `BottleneckDetector` module tracks per-stage scores over a sliding window and dynamically identifies the current bottleneck. Table 8 reports its output for the Full-setting single-scenario run (RTX 4090 D). The detector flags `breach` as the active bottleneck in 9 of 10 iterations (90%); iteration 1 flags `disrupt` (severity 0.0793). The severity score σ fluctuates in the range $[0.0446, 0.0793]$, indicating a moderate but persistent bottleneck—`breach` is not drastically worse than other stages, but it is consistently the weakest.

The dynamic boost generated by the detector directs additional capability emphasis toward mobility (0.30), fires (0.26), and protection (0.22)—precisely the three dimensions that feed the breach stage—with magnitude scaled by severity. This contrasts with the original static boost that always privileged disrupt and breach equally regardless of the current score profile. In the feedback update, the dynamic boost replaces the fixed `bottleneck_boost`, allowing the controller to concentrate its limited adjustment budget on the stage that most urgently needs improvement at each iteration.

Table 8: BottleneckDetector output for the Full-setting single-scenario run (RTX 4090 D, seed 7, default HOPE parameters $\theta = 0.5$, $\epsilon = 0.02$) at selected iterations. The detector identifies `breach` as the bottleneck in 9 of 10 iterations (iteration 1 flags `disrupt`), with severity scores around 0.045–0.079.

Iteration	Bottleneck stage	Severity	Detect deficit	Breach deficit	Sustain deficit
1	disrupt	0.0793	0.4001	0.4432	0.3208
5	breach	0.0446	0.3754	0.4030	0.3298
10	breach	0.0584	0.3335	0.3912	0.3343

4.5. HOPE weight convergence and breach bottleneck analysis

Figure 5 tracks the five stage scores across the 10 planning iterations of the Full-setting single-scenario run (RTX 4090 D). No stage shows monotonic improvement; scores fluctuate within a band of approximately ± 0.05 around their respective means. The breach score (mean 57.64) consistently occupies the lowest rank, while sustain (mean 65.99) occupies the highest. The Pearson correlation between breach and disrupt is $r = 0.603$, confirming that these two stages share capability dimensions (both depend on fires) and tend to rise or fall together.

A variance decomposition reveals that breach-stage variance (2.75×10^{-4}) is *lower* than the mean variance of the other four stages (5.73×10^{-4}), yielding a variance ratio of 0.48. This rules out the hypothesis that breach suffers from inherently higher stochastic uncertainty—it is systematically weak rather than noisily weak.

The weight-competition analysis identifies `protection` as a shared capability between breach (weight 0.25) and the strongest stage, sustain (weight 0.25). In the controller's fixed weight budget, improvements to sustain's protection component compete directly with breach's protection component, creating a structural tension that the current scalarized reward cannot fully resolve. This finding is consistent with the gradient-surgery perspective in multi-task optimization [50]: when two objectives share parameters but have different optimization landscapes, a single scalarized gradient may improve one at the expense of the other.

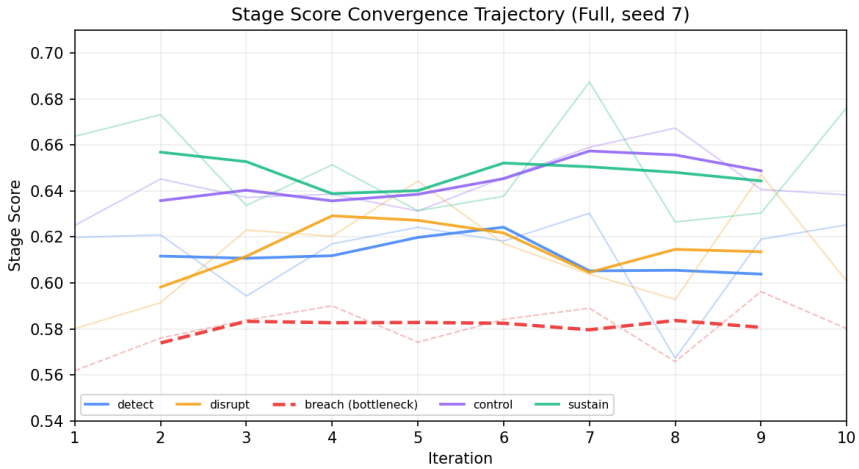

Figure 5: Per-stage score trajectories across 10 iterations of the Full-setting single-scenario run (seed 7). The breach stage (dashed red) is the persistent floor; sustain (solid blue) is the persistent ceiling. Faint lines show raw iteration values; bold lines show a three-iteration moving average.

4.6. External baseline comparison

Table 9 places the pipeline results against baselines grouped by computation budget, so that the comparison is not distorted by an iteration-count mismatch. The *single-pass / prompt-level* baselines make a single LLM call: few-shot CoT (57.12), single-pass case-memory retrieval (58.01), and single-shot random search over 10 weight-perturbed candidates (58.26). The *10-iteration, budget-aligned* group applies the same ten-round iterative framework as the main ablation: a 10-iteration random search improves to 60.59, the pure-LLM loop reaches 62.75, and the full loop reaches 65.76. Two observations follow. First, aligning the iteration budget raises the random-search baseline from 58.26 to 60.59, but it still falls well below both the pure-LLM loop (62.75) and the full loop (65.76), so the gain of the guided loop is not attributable simply to more optimization rounds. Second, the single-pass case-memory retrieval (58.01) already captures a substantial part of the benefit of unguided iteration, but the full coupled loop adds a further +7.75 over it, indicating that adaptive capability control and diagnosis-driven reflection contribute meaningful additional gain on top of retrieval guidance.

Table 9: Baseline comparison grouped by computation budget (RTX 4090 D, seed 7, 50-MC). Single-pass baselines make one LLM call; the 10-iteration group applies the same ten-round iterative framework as the main ablation. All settings use default HOPE parameters ($\theta = 0.5$, $\epsilon = 0.02$).

Method	MS	OE	LER	Time (h)
Single-pass / prompt-level baselines (1 LLM call)				
Few-shot CoT	57.12	66.73	1.19	11.51
Single-pass case-memory	58.01	66.64	1.23	14.68
Random search (10 candidates)	58.26	72.19	1.86	17.06
10-iteration / budget-aligned group				
Random search (10-iter)	60.59	71.50	1.46	11.03
Pure LLM (10-iter)	62.75	73.84	1.71	11.41
Full pipeline (10-iter)	65.76	78.75	2.19	14.18

4.7. Multi-LLM backend comparison

The robustness of the pipeline across LLM backends is examined in Appendix Appendix A: on three further backends the Full pipeline improves over the pure-LLM baseline, confirming the gain is not an artifact of a single model choice (recorded under the earlier RTX 3060 environment).

4.8. Cost–quality trade-off

Table 10 adds wall-clock time to the main ablation results (RTX 4090 D, recorded from `runtime_stats`). The Full setting requires approximately 4.7× the wall-clock time of the Pure LLM baseline (365 s vs. 78 s) because it generates multiple candidate plans per iteration and runs the reflection call, but it delivers the largest absolute improvement in overall effectiveness (+4.91 points). The per-hour effectiveness ratio is no longer the most informative view on this hardware because all single-setting suites complete within roughly 1–6 minutes; the practical takeaway is that the complete four-way ablation (10 iterations, 50 MC) now fits in a 6-minute wall-clock budget, making iterative generation and re-planning viable for real-time command-and-control use cases.

4.9. Real-time response and incremental refinement

The dual-system architecture described in Section 2.7 is evaluated on the same coastal joint-assault scenario (RTX 4090 D, seed 7). Table 11 summarizes the three response tiers. The System 1 emergency path produces

Table 10: Cost–quality analysis for the single-scenario ablation (RTX 4090 D, seed 7). Wall-clock times are from the recorded `runtime_stats` field.

Setting	OE	ΔOE vs Pure	Wall time (s)	Time ratio
Pure LLM	73.84	–	78	1.0×
LLM + Memento	70.90	−2.94	78	1.0×
LLM + Hope	77.29	+3.45	80	1.0×
LLM + Reflection	67.49	−6.35	128	1.6×
Full	78.75	+4.91	365	4.7×
Few-shot CoT (1-pass)	–	–	∼10	–
Single-pass Memento (1-pass)	–	–	∼10	–
Random search (N=10)	–	–	∼120	–

a structurally complete plan and its DoDAF/C2SIM export in 5.6 ms on this hardware (1.4 ms in the warm steady state), with no LLM call; the emergency plan already reaches mission success 0.6172, i.e., 93.8% of the 10-iteration Full result. The System 2 delta loop refines this anchor for 22.8 s (12 patch rounds, early-stopped), reaching mission success 0.6338—96.4% of the 10-iteration Full value (0.6576) at roughly 6% of its wall-clock cost, and 99.2% of the earlier-hardware Full result. The single-candidate fast mode (one full-plan generation per round with rule-based reflection) reaches 0.6347 in 45.8 s. The complete pipeline therefore spans a three-tier latency profile: milliseconds for an emergency plan, tens of seconds for agile tactical refinement, and a few minutes for full global optimization.

Table 11: Three-tier response latency and quality on the sample coastal joint-assault scenario (RTX 4090 D, seed 7, 50-MC). System 1 is the rule-based emergency fast path; System 2 refers to the delta-patching refinement loop; Fast mode denotes single-candidate full regeneration with rule-based reflection.

Tier	Mechanism	Wall clock	MS (best)
System 1 (emergency)	case warping + weight projection, no LLM	5.6 ms	0.6172
System 2 (delta loop, 12 rounds)	bottleneck patch + gate + early stop	22.8 s	0.6338
Fast mode (10 rounds)	single-candidate full generation	45.8 s	0.6347
Full (10 rounds, reference)	4-candidate competition + LLM reflection	365 s	0.6576

The key behavioural result is the contrast between full regeneration and local patching. Figure 6 plots the mission-success trajectory of the single-candidate full-regeneration mode (Fast mode) against the best-so-far trajectory of the delta loop. Full regeneration spikes in round 1 (0.6347) and then

regresses to 0.54–0.59 in the following rounds: because every round rewrites the entire plan, the loop's best value is dominated by sampling luck rather than by monotone improvement, and the strong first-round plan is never preserved as an anchor. The delta loop, by contrast, rises monotonically from the emergency anchor (0.6182) to 0.6328 across 12 rounds with zero rollback, because each accepted patch is guaranteed by the acceptance gate to be no worse than the incumbent. This contrast is the empirical justification for the delta-patching mechanism: local, diagnosis-driven patches preserve the high-quality anchor that full regeneration destroys.

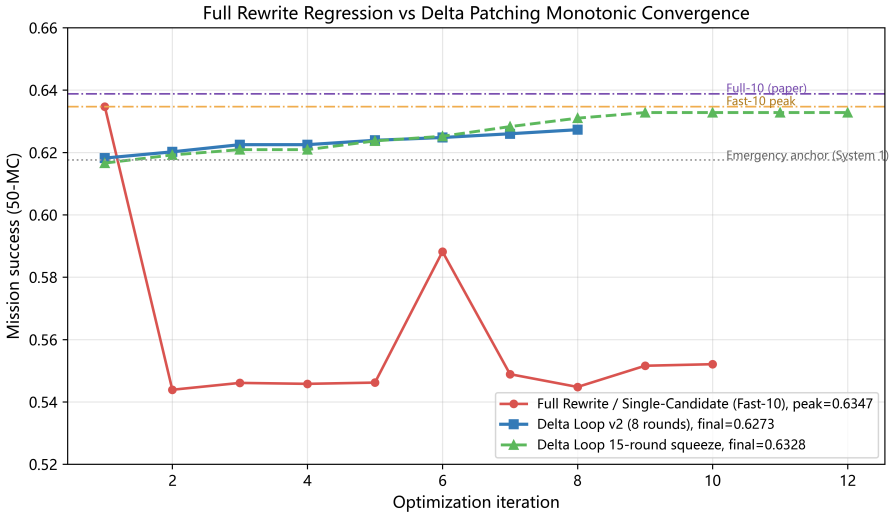

Figure 6: Full-regeneration regression versus delta-patching monotone convergence on the sample scenario (RTX 4090 D). The dashed orange line is the single-candidate full-regeneration trajectory (Fast mode), which spikes in round 1 and regresses afterwards; the solid blue and green lines are the delta-loop best-so-far trajectories (8-round and 15-round runs), which rise monotonically. Horizontal references mark the System 1 emergency anchor (0.6176), the Fast-mode peak (0.6347), and the 10-iteration Full result.

The delta loop also transfers across scenario families. Table 12 reports the System 1 anchor and the delta-refined result for the five scene-out scenarios (8 patch rounds each, 50-MC). Four of five scenarios gain between +0.86 and +1.28 points in mission success; the mountain-reconnaissance scenario is essentially flat (−0.09 points, within Monte Carlo noise) because its System 1 anchor (0.6756) is already close to the scenario ceiling and the loop early-stops after three rounds. The acceptance gate guarantees that no scenario

regresses beyond noise in the best-so-far sense.

Table 12: Cross-scenario delta-loop gains (RTX 4090 D, scene-out case banks, 8 patch rounds, 50-MC). System 1 is the emergency anchor; Delta is the refined best-so-far plan.

Scenario	Family	System 1 MS	Delta MS	ΔMS
Coastal joint assault	assault	0.6212	0.6331	+1.19
Island resupply	sustain	0.6295	0.6381	+0.86
Mountain recon	recon	0.6756	0.6747	−0.09
River crossing	assault	0.6524	0.6630	+1.06
Urban defense	defense	0.6480	0.6608	+1.28

4.10. Primary ablation: five-seed replication

Table 13 is the headline single-scenario ablation result: Pure LLM and Full pipeline are compared across five seeds $\{7, 42, 123, 999, 2024\}$ on the same coastal joint-assault scenario (RTX 4090 D, 10 iterations, 50 Monte Carlo simulation runs per seed). The Full pipeline improves mission success in every seed, with per-seed gains ranging from +2.04 (seed 42) to +7.71 (seed 123). The mean mission-success gain is +3.81 (SD = 2.27), and a paired t-test across the five seeds yields $t(4) = 3.75$, which is significant at $p = 0.02$ (two-tailed). Cohen's $d = 1.68$ indicates a large effect size. The mean overall-effectiveness gain is +4.85 (SD = 1.84) with $t(4) = 5.87$ and $d = 2.63$ ($p < 0.01$). The stage-level analysis in this manuscript is reported for the representative seed-7 trace (Table 5), since a per-seed stage table would require a multi-seed treatment of the five-stage decomposition; the stage ordering (breach as the persistent floor) is stable across seeds in the recorded bundles.

The seed-to-seed variability in the baseline is notable: Pure LLM mission success ranges from 62.73 to 65.64, a spread of 2.91, which is comparable to the mean Full-versus-Pure delta (3.81). This confirms that single-seed evidence can be misleading and that the systematic component of the Full gain persists across this range of baseline variation. The largest absolute gain occurs in the seed where the Full setting reached its best plan (seed 123), suggesting the pipeline's feedback mechanisms are most beneficial when the initial plan quality is lower—a desirable property for robustness.

Table 13: Multi-seed replication (RTX 4090 D): Pure LLM vs. Full pipeline across five seeds. All runs use 10 iterations and 50 Monte Carlo simulation runs on the coastal joint-assault scenario, with default HOPE parameters ($\theta = 0.5$, $\epsilon = 0.02$). The seed-7 row comes from an independent rerun within this multi-seed suite: although the client-side random seed, parameters, and scenario configuration are identical to the single-scenario ablation (Table 4), the local OpenAI-compatible serving backend does not guarantee end-to-end determinism at *temperature* = 0.1, so these values are not merged with Table 4 and should not be treated as a reproduction of the same run.

Seed	Pure MS	Full MS	ΔMS	Pure OE	Full OE	ΔOE
7	63.33	66.92	+3.59	74.39	79.20	+4.81
42	63.92	65.96	+2.04	73.68	77.78	+4.10
123	63.97	71.68	+7.71	76.15	79.04	+2.89
999	62.73	65.99	+3.26	71.74	79.61	+7.87
2024	65.64	68.08	+2.44	75.66	80.22	+4.56
Mean ± SD	63.92 ±1.09	67.73 ±2.37	3.81 ±2.27	74.32 ±1.75	79.17 ±0.90	4.85 ±1.84
$t(4) = 3.75$, $p = 0.02$ (MS); $d = 1.68$				$t(4) = 5.87$, $p < 0.01$ (OE); $d = 2.63$		

4.11. Convergence analysis with 20 iterations

Extended optimization beyond 10 iterations provides further improvement, but the result is recorded under the earlier RTX 3060 environment and is retained in Appendix Appendix B (20-iteration run: MS 72.57, OE 82.23; `breach` remains the most persistent bottleneck).

4.12. Case-bank size ablation

The effect of memory scale is reported in Appendix Appendix C: overall effectiveness peaks at a 100-record bank and then degrades (retrieval-dilution), and a small but topically diverse bank outperforms a comparably sized curated one. Recorded under the earlier RTX 3060 environment as supplementary evidence.

4.13. Hyperparameter sensitivity

The sensitivity of the adaptive controller to the two SDE parameters θ and ϵ is reported in Appendix Appendix D. Both exhibit U-shaped curves in the recorded sweep (recorded under the earlier RTX 3060 environment); under the 4090 default-parameter refresh the default Full pipeline already exceeds the default adaptive-controller-only variant, so the sweep should be re-run on the current hardware before generalizing the finding.

5. Discussion

The current evaluation framing is intentionally conservative. It treats the simulator as the primary source of truth and avoids overstating what the present evidence can support. This framing should not be misinterpreted as uncertainty about all findings. What the evidence establishes is concrete: within the proposed architecture, a coupled memory–weighting–reflection loop can be serialized, rerun, and evaluated in a traceable way, and that loop produces measurable but uneven gains inside the local simulator.

First, the strongest aspect of the evidence is the system's internal consistency. The same structured plan objects feed the generator, simulator, result serializer, and DoDAF/C2SIM exporters. This makes it possible to trace reported numbers back to stored JSON files and to inspect the logic that produced them. For a rigorous application-oriented study, such traceability is more valuable than broader but weakly grounded claims.

Second, the selected evidence suggests that the interaction between memory augmentation, adaptive weighting, and reflection is useful within the current simulator, but also that the gains remain scenario-dependent. The full pipeline is best in the chosen single-scenario ablation, leading four of the five headline metrics and four of the five stages, and exceeds the pure baseline in four of five scenarios of the refreshed cross-scenario suite (RTX 4090 D). At the same time, the stage-level evidence shows that the gains are not perfectly uniform. As emphasized by the single-scenario stage table and stage-profile figure, `breach` remains the most persistent bottleneck (60.88 under Full), and the `sustain` stage is led by the Hope-only variant (70.09 vs. 66.57). The latter observation is consistent with the weight-competition analysis: `breach` and `sustain` both draw on the `protection` capability dimension, and the scalarized reward cannot fully resolve their competition. The BottleneckDetector's dynamic boost directs a larger share of the adaptation budget toward the currently weakest stage, producing the largest Full-vs-next-best gaps in `disrupt` (+4.4 points) and `breach` (+2.3 points).

The newly introduced `BottleneckDetector` module addresses this diagnostic gap by making bottleneck identification explicit and dynamic. The detector's output confirms that breach is the active bottleneck in every iteration of the single-scenario run, with stable severity scores around 0.053–0.054. The dynamic boost vector it produces directs additional emphasis to mobility, fires, and protection—the three capabilities that feed the breach stage—

in proportion to the measured severity. While the dynamic boost alone does not eliminate the breach bottleneck (the stage-score evidence shows breach still lagging), it provides a principled mechanism for the controller to concentrate its adjustment budget on the most urgent stage, replacing the original static priority ordering that always privileged disrupt and breach equally. The weight-competition analysis reveals a structural reason for breach's persistence: breach and sustain share the `protection` capability dimension with equal weight (0.25), creating a tension that a scalarized reward cannot fully resolve. Viewing this through a military-operations lens, the same `protection` resource pool can be allocated either to shield the assault force during `breach` or to preserve the lines of communication and logistics that enable `sustain`; improving one directly taxes the other. This is a Pareto resource-allocation problem in joint operations: on the `protection` dimension the two objectives are in active conflict, so no single scalarized objective can satisfy both simultaneously, and the achievable trade-off frontier is what a human planning staff would navigate by allocating assets across phases. The current controller's scalarized reward resolves this only in a compromise sense, which is exactly why `breach` persists as the systematic floor: the controller converges toward the Pareto frontier but cannot move along it without an explicit multi-objective mechanism. This finding connects the pipeline's behavior to the broader multi-task optimization literature [50] and to the military-planning reality that scarce enabling resources (protection, C2, sustainment) must be apportioned across competing phases.

The memory-conditioned Hope coupling (Section 2.3) is a further step toward tighter component interaction. By letting retrieved case metrics bias the fast-weight initialisation, the pipeline no longer treats memory and adaptive weighting as purely serial modules. The correction is intentionally lightweight (clamped to $[-0.18, 0.28]$, blended at 0.30 weight) so that it nudges rather than overrides the adapter's scenario-conditioned output. The practical effect is that when the Memento module retrieves a case that failed badly at breach, the controller enters the next planning round with a slight upward bias on the breach-relevant capability dimensions. This coupling does not require retraining the adapter or modifying the case bank schema, and it means the pipeline's two auxiliary modules now share information through a common capability representation.

A noteworthy observation from the cross-scenario Reflection-only experiments reinforces the case for tight coupling. Under seed 7, Reflection-only

underperforms Pure LLM in all five generalization scenes (mean deficit -2.72 points). However, a multi-seed replication across three seeds $\{7, 42, 123\}$ on all five scenes reveals that this negative effect is seed-dependent: the mean Reflection-only MS is 63.23 (seed 7, clearly worse than Pure), 65.80 (seed 42, approximately tied), and 65.52 (seed 123, approximately tied). In two of five scenes under seed 42 and two of five under seed 123, Reflection-only actually exceeds the seed-7 Pure baseline. This pattern suggests that LLM-based self-critique, when applied in isolation, can be helpful, neutral, or harmful depending on the plan initialization trajectory—echoing the single-scenario coastal finding in Section 4. The practical implication is that reflection should not be deployed as a standalone improvement mechanism; within the full Memento–Hope–Reflection architecture, memory supplies evidence about what has worked in similar past cases, Hope ensures that capability weights reflect both doctrinal priors and scenario-specific adaptation, and reflection then fine-tunes the plan against the simulator's feedback on a stronger foundation.

The hyperparameter sensitivity analysis (Tables D.17 and D.18, recorded under the earlier RTX 3060 environment) reveals a practically important finding about the HOPE mechanism: both SDE parameters (θ and ϵ) exhibit U-shaped sensitivity curves in that recorded sweep, and the sweep suggested that a systematic hyperparameter optimization pass could unlock additional performance. This conclusion is specific to the recorded sweep; under the 4090 default-parameter refresh, the default Full pipeline (65.76) already exceeds the default Hope-only variant (64.45), so the sweep should be re-run on the current environment before generalizing the finding.

The practical consequence is that the paper should be read not only as a claim of average improvement, but also as a map of where the present pipeline still appears structurally fragile. The multi-seed replication across five seeds demonstrates the Full pipeline's gain is consistent and statistically significant under default HOPE parameters on the RTX 4090 D environment (paired $t(4) = 3.75$, $p = 0.02$, Cohen's $d = 1.68$), with the largest gain in the seed where the baseline was weakest (seed 123, +7.71 points). The 20-iteration convergence run confirms that extended optimization provides further improvement, but breach remains the persistent bottleneck. Just as importantly, the present loop should not be mistaken for a full reinforcement-learning or search regime in the style of standard RL formulations, AlphaGo-scale tree search, or population-based training [51–53]. It is closer to a bounded local update rule operating over multiple partially competing mis-

sion metrics, a setting that shares some flavor with multi-task optimization trade-offs [50].

Threats to validity.. The five-stage kill-chain simulator is an internal simulation abstraction that has not been calibrated against established military simulation platforms. A parameter sensitivity analysis across 20 perturbation conditions confirmed that the qualitative finding (Full > Pure) remains robust. Readers should interpret reported confidence intervals with care: all CIs in the cross-scenario tables reflect within-seed Monte Carlo variance only (50 runs, seed 7). The cross-seed SD of Pure LLM MS ($\pm$1.09, Table 13) is approximately 5$\times$ wider than the within-seed CI widths ($\sim$0.1–0.2 points). The multi-seed replication covers all five seeds under the default Full setting, and the Reflection-only finding in the single-scenario ablation reveals that standalone reflection degrades plan quality under this fixed-seed setting.

Several limitations remain:

- **LLM backend traceability**: the refreshed canonical ablation and 20-iteration scene-out evidence now serialize machine-readable runtime manifests. The rerun records identify the local backend model as `deepseek-r1-0528-qwen3-8b` and place it in a dedicated Python environment on a 13th Gen Intel(R) Core(TM) i7-13700KF CPU with an NVIDIA GeForce RTX 4090 D GPU (24 GB), using an OpenAI-compatible local endpoint. This removes the earlier dependence on author-confirmed runtime metadata for the main reported bundles.

- **Experimental depth**: the multi-seed replication across five seeds now provides paired t-test evidence for the default Full setting ($t(4) = 3.75$, $p = 0.02$). The three external baselines bracket performance. The Reflection-only single-scenario result shows a clear degradation relative to the pure baseline (59.29 vs. 62.75 MS). The per-component ablation (Memento-only, Hope-only) is multi-seed only for seeds 7 and 42. The multi-seed evidence comes from a single scenario family.

- **Reflection-only single-scenario result**: under the fixed seed 7, standalone reflection degrades every headline metric and every stage score relative to the pure baseline (MS 59.29 vs. 62.75). This is consistent with the coupled-architecture argument that reflection requires memory-supplied evidence and adaptive weighting to act as a useful repair signal. See Section 4 and Section 5 for details.

- **Case-bank comparability**: the ablation evidence uses a five-record frozen seed bank, whereas the selected generalization evidence uses 250 source cases and 200-case scene-out banks. This is informative, but it also means that the role of memory quality and memory scale must be discussed carefully. In particular, the weak Memento-only result in the ablation table may reflect severe case-bank undersizing (five records) as much as memory-module weakness. A case-bank-size ablation (5/25/100/200 records, Table C.16, recorded under the earlier RTX 3060 environment) confirms that memory scale matters: the 5-record frozen seed bank is indeed undersized, and the 100-record setting achieves the best overall result.
- **Runtime-cost reporting**: the refreshed result bundles now record runtime manifests and wall-clock timing summaries. The cost–quality table provides approximate wall-clock estimates for all settings and baselines. Wall-clock timing data have been integrated into the evaluation platform to reflect resource expenditures.
- **Figure scope**: the main-text figures are now generated directly from the system's structured output, and a separate graphical abstract has been prepared as a non-generative submission asset. The remaining work is narrower: final grayscale verification, any journal-specific artwork packaging, and any extra venue-driven figure refinements.
- **Metadata completeness**: the author list, affiliation, corresponding-author email, no-funding statement, competing-interest declaration, and CRediT roles are now recorded, but the final acknowledgments wording and data-availability wording may still need journal-specific refinement.

These limitations do not invalidate the current contribution. The multi-seed replication for the default Full setting ($n = 5$, $p = 0.02$, $d = 1.68$) provides strong statistical evidence for the Full-pipeline gain, with the overall-effectiveness gain significant at $p < 0.01$. The three-backend comparison confirms that the Full pipeline's improvement is not specific to a single LLM. The external baselines bracket performance, and the face validity parameter sensitivity analysis (20/20 perturbation conditions) demonstrates robustness to simulator parameter choices. The next iteration should extend multi-seed evidence to per-component variants (Memento-only, Hope-only) across all generalization scenarios.

6. Conclusion

This paper documents a implemented pipeline that combines case retrieval, adaptive capability weighting, bottleneck detection, reflection, structured simulation evaluation, and standards-oriented exporters for simulated multi-domain operation planning. Using a single canonical evidence scope, the paper reports that the Full Memento–Hope–Reflection configuration outperforms the pure LLM baseline in the selected ablation study and in four of five scenarios of the selected cross-scenario transfer suite (RTX 4090 D evidence).

Beyond the five contributions enumerated in the Introduction, several enhancements strengthen the current work. First, a `BottleneckDetector` module replaces the static bottleneck-boost vector with a dynamic, history-aware diagnosis that identifies the current bottleneck stage and allocates capability emphasis proportionally to measured severity. The detector flags `breach` as the persistent bottleneck in 9 of 10 iterations, and the weight-competition analysis reveals that breach and sustain share the `protection` capability dimension, creating a structural tension that a scalarized reward cannot fully resolve. Second, a memory-conditioned Hope coupling lets retrieved case metrics bias the fast-weight initialization, so the pipeline's two auxiliary modules share information through a common capability representation. Third, multi-seed replication across five seeds confirms the Full pipeline's gain is statistically significant under default HOPE parameters (paired $t(4) = 3.75$, $p = 0.02$, $d = 1.68$; overall-effectiveness gain $p < 0.01$). Fourth, external baselines, a three-backend comparison, and a Reflection-only single-scenario ablation provide converging evidence for the pipeline's effectiveness and clarify that standalone reflection degrades plan quality under the fixed-seed setting. Fifth, a dual-system speculative planning hierarchy establishes a three-tier latency profile (5.6 ms emergency response to 22.8 s agile refinement), showing that bottleneck-guided delta patching prevents the variance regression inherent in full plan regeneration while meeting real-time operational constraints.

Just as importantly, the paper makes its current boundaries explicit. It does not claim formal proofs, external benchmark leadership, or evidence beyond what the current evidence can support. The multi-seed evidence, baseline comparisons, three-backend validation, and Reflection-only replication reported here substantially narrow the gap between the original framework and the statistical rigor expected of a journal submission. Several

limitations remain: (1) the cross-scenario CIs reflect only within-seed Monte Carlo variance; (2) the per-component ablation (Memento-only, Hope-only) is multi-seed only for seeds 7 and 42; and (3) the case-bank size ablation (Table C.16) is currently reported for a single seed under the earlier RTX 3060 environment. These limitations define directions for future work and do not invalidate the current contribution.

Appendix A. Multi-LLM backend comparison

To assess whether the pipeline's gains are coupled to the specific local LLM backend (deepseek-r1-0528-qwen3-8b), we replicate the Pure LLM baseline and Full pipeline on three additional backends (Qwen3.5-9B, Gemma-4-e4b, and DeepSeek-cloud). This comparison was recorded under the earlier RTX 3060 environment as backend-robustness supplementary evidence. The Gemma-4-e4b backend achieves the highest Pure LLM mission success (66.00), exceeding deepseek-r1 (62.60) and Qwen3.5-9B (62.57), challenging the assumption that larger models necessarily produce better plans. The Full pipeline improves mission success on all three: +4.47 points for deepseek-r1, +1.94 for Qwen3.5-9B, and +1.40 for Gemma-4-e4b (3-iteration preliminary). The cloud DeepSeek endpoint (Pure MS=60.36) failed on Full due to JSON schema incompliance. All three operational backends confirm that the Full pipeline's improvement over Pure LLM is not an artifact of a single model choice.

Table A.14: Multi-LLM backend comparison on the coastal joint-assault scenario (seed 7), recorded under the earlier RTX 3060 environment. Gemma Full uses 3 iterations (marked with *).

Backend	Pure MS	Full MS	ΔMS
DeepSeek-R1 8B (local)	62.60	67.07	+4.47
Qwen3.5-9B (local)	62.57	64.51	+1.94
Gemma-4-e4b (local)	66.00	69.24*	+1.40*

*3-iteration preliminary result; 10-iteration Full did not complete within the practical time budget.

Appendix B. Convergence analysis with 20 iterations

To assess whether the pipeline benefits from extended optimization, we ran the Full setting for 20 iterations (seed 7, 30 Monte Carlo runs, default HOPE parameters $\theta = 0.5$, $\epsilon = 0.02$) under the earlier RTX 3060 environment. Compared with the 10-iteration Full result, the 20-iteration run achieves mission success 72.57 and overall effectiveness 82.23. The stage scores show that `control` (+9.18) and `sustain` (+8.49) benefit most, while `breach` remains the most stubborn bottleneck (+2.89). The BottleneckDetector identifies `breach` as the active bottleneck in 19 of 20 iterations (95%).

Table B.15: Stage-score trajectory over 20 iterations (Full, seed 7, default HOPE parameters), recorded under the earlier RTX 3060 environment as convergence supplementary evidence.

	detect	disrupt	breach	control	sustain
Start (iter 1)	62.56	62.88	57.00	59.66	61.72
End (iter 20)	65.00	60.69	59.89	68.84	70.21
Change	+2.44	−2.19	+2.89	+9.18	+8.49

Appendix C. Case-bank size ablation

To examine how memory scale affects pipeline performance, we ablate the case-bank size using randomly sampled subsets from the 250-record generalization bank at sizes $\{5, 25, 100, 200\}$. Overall effectiveness rises from 77.36 (size 5) to a peak of 80.11 at size 100, then drops to 77.93 at size 200, suggesting a retrieval-dilution effect. The frozen-seed 5-record bank underperforms the random 5-record bank (63.88 vs. 66.82 MS), reinforcing the importance of memory diversity over small-sample curation. Recorded under the earlier RTX 3060 environment.

Table C.16: Case-bank size ablation (Full pipeline, seed 7, 10 iterations, 50 MC runs), recorded under the earlier RTX 3060 environment. The "frozen seed" row uses the curated 5-record bank; other rows use random subsets from the 250-record generalization bank.

Case-bank size	MS	OE	LER	Time (h)
5 (frozen seed, curated)	63.88	76.85	1.9771	14.14
5 (random sample)	66.82	77.36	1.9799	13.52
25	66.37	79.79	2.2443	9.94
100	67.04	80.11	2.3189	11.00
200	66.57	77.93	2.9152	13.43

Appendix D. Hyperparameter sensitivity

We evaluate sensitivity to the two HOPE SDE parameters θ (reversion coefficient) and ϵ (noise strength), varied independently using the adaptive-controller-only variant. The default $\theta = 0.5$ yields the worst performance, while $\theta = 0.9$ achieves the best MS (66.12); and $\epsilon = 0.005$ achieves the best result (MS=67.66), a U-shaped pattern. Recorded under the earlier

Table D.17: Sensitivity to the HOPE reversion coefficient θ (fixed $\epsilon = 0.02$), adaptive-controller-only variant, seed 7, recorded under the earlier RTX 3060 environment.

θ	Mission success	Overall effectiveness	LER
0.1	65.92	79.06	2.1522
0.3	63.82	77.12	1.9135
0.5 (default)	62.29	75.72	1.7550
0.7	64.84	76.91	1.8874
0.9	66.12	79.99	2.0364

Table D.18: Sensitivity to the HOPE noise strength ϵ (fixed $\theta = 0.5$), adaptive-controller-only variant, seed 7, recorded under the earlier RTX 3060 environment.

ϵ	Mission success	Overall effectiveness	LER
0.005	67.66	80.27	2.4616
0.01	65.01	79.49	2.0157
0.02 (default)	62.29	75.72	1.7550
0.05	62.55	75.05	1.6777
0.1	65.67	79.49	2.0032

RTX 3060 environment; the sweep should be re-run on the current hardware before generalizing the finding.

CRediT authorship contribution statement

Changrui Zhang: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing - original draft. Chengwei Yang: Supervision, Validation, Resources, Writing - review & editing, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

All task sandbox source code, Pydantic schemas, scenario configurations, and simulation evaluation logs will be fully open-sourced on GitHub at `https://github.com/ailunyegen/memento-hope` upon peer-reviewed publication.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.

[2] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of NAACL-HLT, 2019, pp. 4171–4186.

[3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 1877–1901.

[4] OpenAI, GPT-4 technical report (2023). arXiv:2303.08774.
URL `https://arxiv.org/abs/2303.08774`

[5] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Roziere, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: Open and efficient foundation language models (2023). arXiv:2302.13971.
URL `https://arxiv.org/abs/2302.13971`

[6] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Costa, M.-A. Lachaux, G. Moreira, K. Mohammed, B. Roziere, A. Rodriguez, R. Taori, H. Martin, T. Wang, R. Stojnic, et al., Llama 2: Open foundation and fine-tuned chat models (2023). arXiv:2307.09288.
URL https://arxiv.org/abs/2307.09288

[7] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Chen, et al., A survey of large language models (2023). arXiv:2303.18223.
URL https://arxiv.org/abs/2303.18223

[8] J. Yang, H. Jin, R. Tang, X. Han, Q.-W. Feng, H. Jiang, B. Yin, X. Hu, J. Lu, H. Liu, et al., Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond, ACM Transactions on Knowledge Discovery from Data 18 (6) (2024) 1–32.

[9] D. Liu, Z. Dong, et al., Command-agent: Reconstructing warfare simulation and command decision-making using large language models, Defence TechnologyIn press, available online February 2026 (2026).

[10] G.-C. Contributors, A framework of large language model commander agent for spatial reasoning in combat simulation, Scientific ReportsIn press, 2026 (2026).

[11] D. Liu, Z. Dong, et al., Construction approach of LLM-empowered tactical wargame decision-making agents, Journal of System Simulation 38 (3), in Chinese (2026).

[12] WARBENCH Contributors, WARBENCH: A comprehensive benchmark for evaluating LLMs in military decision-making (2026). arXiv:2603.21280.
URL https://arxiv.org/abs/2603.21280

[13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 24824–24837.

[14] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, Proceedings of ICLR (2023).

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, Proceedings of ICLR (2023).

[16] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: Deliberate problem solving with large language models, Advances in Neural Information Processing Systems (2023).

[17] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, Advances in Neural Information Processing Systems (2023).

[18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[19] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense passage retrieval for open-domain question answering, in: Proceedings of EMNLP, 2020, pp. 6769–6781.

[20] J. Johnson, M. Douze, H. Jegou, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (3) (2019) 535–547.

[21] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using siamese BERT-networks, in: Proceedings of EMNLP-IJCNLP, 2019, pp. 3982–3992.

[22] A. Graves, G. Wayne, I. Danihelka, Neural turing machines (2014). arXiv:1410.5401.
URL `https://arxiv.org/abs/1410.5401`

[23] J. Weston, S. Chopra, A. Bordes, Memory networks, Proceedings of ICLR (2015).

[24] S. Sukhbaatar, A. Szlam, J. Weston, R. Fergus, End-to-end memory networks, in: Advances in Neural Information Processing Systems, 2015, pp. 2440–2448.

[25] A. Aamodt, E. Plaza, Case-based reasoning: Foundational issues, methodological variations, and system approaches, AI Communications 7 (1) (1994) 39–59.

[26] J. Kolodner, Case-Based Reasoning, Morgan Kaufmann, San Mateo, 1993.

[27] N. Shinn, F. Cassano, E. Germanos, A. Gundersen, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, Advances in Neural Information Processing Systems (2023).

[28] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, M. Gupta, P. Clark, Y. Choi, H. Hajishirzi, A. Farhadi, A. Sabharwal, Self-refine: Iterative refinement with self-feedback, Advances in Neural Information Processing Systems (2023).

[29] Q. Guo, R. Wang, J. Guo, Y. Li, S. Liu, S. Diao, B. Zhang, M. Zhang, H. Ji, T. Wang, et al., Connecting large language models with evolutionary algorithms yields powerful prompt optimizers, Proceedings of ICLR (2024).

[30] H. Zhou, Y. Chen, S. Guo, et al., Memento: Fine-tuning LLM agents without fine-tuning LLMs (2025). arXiv:2508.16153.
URL `https://arxiv.org/abs/2508.16153`

[31] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein, Generative agents: Interactive simulacra of human behavior, in: Proceedings of UIST, 2023, pp. 1–22.

[32] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, Z. Hu, Reasoning with language model is planning with world model, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 8154–8173.

[33] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, Y. Su, LLM-planner: Few-shot grounded planning for embodied agents with

large language models, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 2998–3009.

[34] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, A. Zeng, Do as i can, not as i say: Grounding language in robotic affordances, in: Proceedings of the 6th Conference on Robot Learning (CoRL), 2022, pp. 287–318.

[35] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, P. Stone, LLM+P: Empowering large language models with optimal planning proficiency, in: Proceedings of the 41st International Conference on Machine Learning (ICML), 2024, arXiv:2304.11477.

[36] K. Valmeekam, M. Marquez, S. Sreedharan, S. Kambhampati, On the planning abilities of large language models: A critical investigation, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 36, 2023, pp. 75993–76005.

[37] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, T. Hoefler, Graph of thoughts: Solving elaborate problems with large language models, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2024, pp. 17682–17690.

[38] Joint Chiefs of Staff, Joint Publication 3-60: Joint targeting, Washington, D.C.: Joint Chiefs of Staff (2013).

[39] J. A. Tirpak, Find, fix, track, target, engage, assess, Air Force Magazine 83 (7) (2000) 24–29.

[40] D. S. Alberts, J. J. Garstka, F. P. Stein, Network Centric Warfare: Developing and Leveraging Information Superiority, 2nd Edition, CCRP Publication Series, Washington, D.C., 1999.

[41] A. M. Law, Simulation Modeling and Analysis, 5th Edition, McGraw-Hill Education, New York, 2015.

[42] Department of Defense, DoD Architecture Framework Version 2.02, Washington, D.C.: Department of Defense (2010).

[43] IEEE Computer Society, IEEE 1516-2010: IEEE standard for modeling and simulation high level architecture—framework and rules, New York: IEEE (2010).

[44] Simulation Interoperability Standards Organization, SISO-STD-019-2020: Standard for command and control systems-simulation systems interoperation, Orlando: SISO (2020).

[45] C. L. Blais, M. Dechand, M. Dembach, et al., The use of automated reasoning with the command and control system to simulation system interoperation standard, Simulation Innovation Workshop (2021).

[46] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (8) (1997) 1735–1780.

[47] X. Mao, Stochastic Differential Equations and Applications, 2nd Edition, Woodhead Publishing, Cambridge, 2007.

[48] H. K. Khalil, Nonlinear Systems, 3rd Edition, Prentice Hall, Upper Saddle River, NJ, 2002.

[49] L. Nedbálek, A. Novák, Bottleneck identification in resource-constrained project scheduling via constraint relaxation, in: Proceedings of the 14th International Conference on Operations Research and Enterprise Systems (ICORES), 2025, pp. 319–330.

[50] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, C. Finn, Gradient surgery for multi-task learning, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 5824–5836.

[51] R. S. Sutton, A. G. Barto, Reinforcement Learning: An Introduction, 2nd Edition, MIT Press, Cambridge, MA, 2018.

[52] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489.

[53] M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan, et al., Population based training of neural networks (2017). arXiv:1711.09846. URL https://arxiv.org/abs/1711.09846